Lecture 10: Wrapping up language models, hidden Markov models

TTIC 31220: Unsupervised Learning and Data Analysis

Instructor: Karen Livescu

TTI-Chicago

April 22, 2026

Announcements

- **Reminder: Project guidelines, timeline** posted under "final project" module on canvas site
  - Anyone having trouble following links in the guidelines?
- **Looking for a project partner?** See announcement about partner survey. Will post survey responses soon.
- **Still need your graded Quiz 1 returned?** See me right after class.

Language models

Evaluating LMs: Cross-entropy

- If X is a discrete random variable taking one of N values with probabilities $p_1, \ldots, p_N$, respectively, then the entropy of X is

$$H(X) = -\sum_{i=1}^{N} p_i \log_2 p_i$$

Evaluating LMs: Cross-entropy

- If X is a discrete random variable taking one of N values with probabilities $p_1, \ldots, p_N$, respectively, then the entropy of X is
$$H(X) = -\sum_{i=1}^{N} p_i \log_2 p_i$$
- The *cross-entropy* of a model $\hat{p}(X)$ with respect to some (i.i.d.) data set $\mathbf{X} = \{x_1, \ldots, x_n\}$ is $H_{\hat{p}}(\mathbf{X}) = -\frac{1}{n} \log_2 \hat{p}(\mathbf{X}) = -\sum_{i=1}^{N} q_i \log_2 p_i$
- Where $q_i = \text{count}(i)/n$ is the empirical distribution of the data set

Evaluating LMs: Cross-entropy

- If X is a discrete random variable taking one of N values with probabilities $p_1, \ldots, p_N$, respectively, then the entropy of X is
$$H(X) = -\sum_{i=1}^{N} p_i \log_2 p_i$$
- The *cross-entropy* of a model $\hat{p}(X)$ with respect to some (i.i.d.) data set $\mathbf{X} = \{x_1, \ldots, x_n\}$ is $H_{\hat{p}}(\mathbf{X}) = -\frac{1}{n} \log_2 \hat{p}(\mathbf{X}) = -\sum_{i=1}^{N} q_i \log_2 p_i$
- Where $q_i = \text{count}(i)/n$ is the empirical distribution of the data set
- For an autoregressive LM $\hat{p}(\cdot)$ on a test set $\mathbf{w} = (w_1, \ldots, w_n)$, w_i are not i.i.d. but they are i.i.d. given their histories:

$$\begin{aligned} H_{\hat{p}}(\mathbf{w}) &= -\frac{1}{n} \log_2 \hat{p}(\mathbf{w}) \\ &= -\frac{1}{n} \sum_{i=1}^{n} \log_2 \hat{p}(w_i | \phi(h_i)) \end{aligned}$$

Evaluating LMs: Perplexity

- Perplexity is related to the cross-entropy via

$$PP_{\hat{p}}(\mathbf{X}) = 2^{H_{\hat{p}}(\mathbf{X})} = 2^{-\frac{1}{n}\log_2 \hat{p}(\mathbf{X})} = \left(\frac{1}{\hat{p}(\mathbf{X})}\right)^{1/n}$$

Evaluating LMs: Perplexity

- Perplexity is related to the cross-entropy via

$$PP_{\hat{p}}(\mathbf{X}) = 2^{H_{\hat{p}}(\mathbf{X})} = 2^{-\frac{1}{n}\log_2 \hat{p}(\mathbf{X})} = \left(\frac{1}{\hat{p}(\mathbf{X})}\right)^{1/n}$$

- Given a test set $\mathbf{w} = (w_1, \ldots, w_n)$ and an autoregressive language model $\hat{p}(w_i|\phi(h_i))$,

$$\begin{aligned} PP_{\hat{p}}(\mathbf{w}) &= 2^{-\frac{1}{n}\sum_{i=1}^{n}\log_2 \hat{p}(w_i|\phi(h_i))} \\ &= \left(\prod_{i=1}^{n}\frac{1}{\hat{p}(w_i|\phi(h_i))}\right)^{1/n} \end{aligned}$$

Evaluating LMs: Perplexity

- Perplexity is related to the cross-entropy via

$$PP_{\hat{p}}(\mathbf{X}) = 2^{H_{\hat{p}}(\mathbf{X})} = 2^{-\frac{1}{n}\log_2 \hat{p}(\mathbf{X})} = \left(\frac{1}{\hat{p}(\mathbf{X})}\right)^{1/n}$$

- Given a test set $\mathbf{w} = (w_1, \ldots, w_n)$ and an autoregressive language model $\hat{p}(w_i|\phi(h_i))$,

$$\begin{aligned} PP_{\hat{p}}(\mathbf{w}) &= 2^{-\frac{1}{n}\sum_{i=1}^{n}\log_2 \hat{p}(w_i|\phi(h_i))} \\ &= \left(\prod_{i=1}^{n}\frac{1}{\hat{p}(w_i|\phi(h_i))}\right)^{1/n} \end{aligned}$$

- So perplexity is the *geometric mean of the per-word inverse probability*

Evaluating LMs: Perplexity

- Perplexity is related to the cross-entropy via

$$PP_{\hat{p}}(\mathbf{X}) = 2^{H_{\hat{p}}(\mathbf{X})} = 2^{-\frac{1}{n}\log_2 \hat{p}(\mathbf{X})} = \left(\frac{1}{\hat{p}(\mathbf{X})}\right)^{1/n}$$

- Given a test set $\mathbf{w} = (w_1, \ldots, w_n)$ and an autoregressive language model $\hat{p}(w_i|\phi(h_i))$,

$$\begin{aligned} PP_{\hat{p}}(\mathbf{w}) &= 2^{-\frac{1}{n}\sum_{i=1}^{n}\log_2 \hat{p}(w_i|\phi(h_i))} \\ &= \left(\prod_{i=1}^{n}\frac{1}{\hat{p}(w_i|\phi(h_i))}\right)^{1/n} \end{aligned}$$

- So perplexity is the *geometric mean of the per-word inverse probability*
- Lower entropy/perplexity $\Rightarrow$ lower uncertainty about the following word $\Rightarrow$ better language model

Perplexity of a distribution vs. perplexity as an evaluation metric

- We have talked about *perplexity of a model evaluated on a test set*
- Another notion is *perplexity of a distribution* (as in t-SNE)

Perplexity of a distribution vs. perplexity as an evaluation metric

- We have talked about *perplexity of a model evaluated on a test set*
- Another notion is *perplexity of a distribution* (as in t-SNE)
- If X is a discrete random variable taking one of N values with probabilities $p_1, \ldots, p_N$, respectively, then the entropy of X is

$$H(X) = -\sum_{i=1}^{N} p_i \log_2 p_i$$

- The perplexity of X is $PP(X) = 2^{H(X)}$

Perplexity of a distribution vs. perplexity as an evaluation metric

- We have talked about *perplexity of a model evaluated on a test set*
- Another notion is *perplexity of a distribution* (as in t-SNE)
- If X is a discrete random variable taking one of N values with probabilities $p_1, \ldots, p_N$, respectively, then the entropy of X is

$$H(X) = -\sum_{i=1}^{N} p_i \log_2 p_i$$

- The perplexity of X is $PP(X) = 2^{H(X)}$
- Intuition: Perplexity of an LM, viewed as a distribution, is the average number of words/tokens possible after a given history (the average *branching factor* of the LM)

Perplexity of a distribution vs. perplexity as an evaluation metric

- We have talked about *perplexity of a model evaluated on a test set*
- Another notion is *perplexity of a distribution* (as in t-SNE)
- If X is a discrete random variable taking one of N values with probabilities $p_1, \ldots, p_N$, respectively, then the entropy of X is

$$H(X) = -\sum_{i=1}^{N} p_i \log_2 p_i$$

- The perplexity of X is $PP(X) = 2^{H(X)}$
- Intuition: Perplexity of an LM, viewed as a distribution, is the average number of words/tokens possible after a given history (the average *branching factor* of the LM)
- Q: What is the perplexity of a uniform LM? A: The vocab size
- Q: What is the minimum perplexity possible? A: 1

Perplexity of a distribution vs. perplexity as an evaluation metric (2)

- **Perplexity as an evaluation metric** is related to the cross-entropy via $PP_{\hat{p}}(\mathbf{w}) = 2^{H_{\hat{p}}(\mathbf{w})}$
- **Perplexity as an evaluation metric** is the perplexity of a **model distribution** with respect to a **data set** (= an empirical distribution)
- The **perplexity of a distribution** $p(X)$ is just $2^{H_p(X)}$
- (The former should ideally have been called "cross-perplexity"...)

Perplexity of a distribution vs. perplexity as an evaluation metric (2)

- **Perplexity as an evaluation metric** is related to the cross-entropy via $PP_{\hat{p}}(\mathbf{w}) = 2^{H_{\hat{p}}(\mathbf{w})}$
- **Perplexity as an evaluation metric** is the perplexity of a **model distribution** with respect to a **data set** (= an empirical distribution)
- The **perplexity of a distribution** $p(X)$ is just $2^{H_p(X)}$
- (The former should ideally have been called "cross-perplexity"...)
  - What is the max of perplexity as an eval metric?

Perplexity of a distribution vs. perplexity as an evaluation metric (2)

- **Perplexity as an evaluation metric** is related to the cross-entropy via $PP_{\hat{p}}(\mathbf{w}) = 2^{H_{\hat{p}}(\mathbf{w})}$
- **Perplexity as an evaluation metric** is the perplexity of a **model distribution** with respect to a **data set** (= an empirical distribution)
- The **perplexity of a distribution** $p(X)$ is just $2^{H_p(X)}$
- (The former should ideally have been called "cross-perplexity"...)
  - What is the max of perplexity as an eval metric? (infinity)

Perplexity of a distribution vs. perplexity as an evaluation metric (2)

- **Perplexity as an evaluation metric** is related to the cross-entropy via $PP_{\hat{p}}(\mathbf{w}) = 2^{H_{\hat{p}}(\mathbf{w})}$
- **Perplexity as an evaluation metric** is the perplexity of a **model distribution** with respect to a **data set** (= an empirical distribution)
- The **perplexity of a distribution** $p(X)$ is just $2^{H_p(X)}$
- (The former should ideally have been called "cross-perplexity"...)
  - What is the max of perplexity as an eval metric? (infinity)
  - What is the perplexity of a uniform model distribution evaluated on an arbitrary data set?

Perplexity of a distribution vs. perplexity as an evaluation metric (2)

- **Perplexity as an evaluation metric** is related to the cross-entropy via $PP_{\hat{p}}(\mathbf{w}) = 2^{H_{\hat{p}}(\mathbf{w})}$
- **Perplexity as an evaluation metric** is the perplexity of a **model distribution** with respect to a **data set** (= an empirical distribution)
- The **perplexity of a distribution** $p(X)$ is just $2^{H_p(X)}$
- (The former should ideally have been called "cross-perplexity"...)
  - What is the max of perplexity as an eval metric? (infinity)
  - What is the perplexity of a uniform model distribution evaluated on an arbitrary data set? (Still the cardinality = vocabulary size)

Perplexity of a distribution vs. perplexity as an evaluation metric (2)

- **Perplexity as an evaluation metric** is related to the cross-entropy via $PP_{\hat{p}}(\mathbf{w}) = 2^{H_{\hat{p}}(\mathbf{w})}$
- **Perplexity as an evaluation metric** is the perplexity of a **model distribution** with respect to a **data set** (= an empirical distribution)
- The **perplexity of a distribution** $p(X)$ is just $2^{H_p(X)}$
- (The former should ideally have been called "cross-perplexity"...)
  - What is the max of perplexity as an eval metric? (infinity)
  - What is the perplexity of a uniform model distribution evaluated on an arbitrary data set? (Still the cardinality = vocabulary size)
  - How do we compare LMs with different vocabularies?

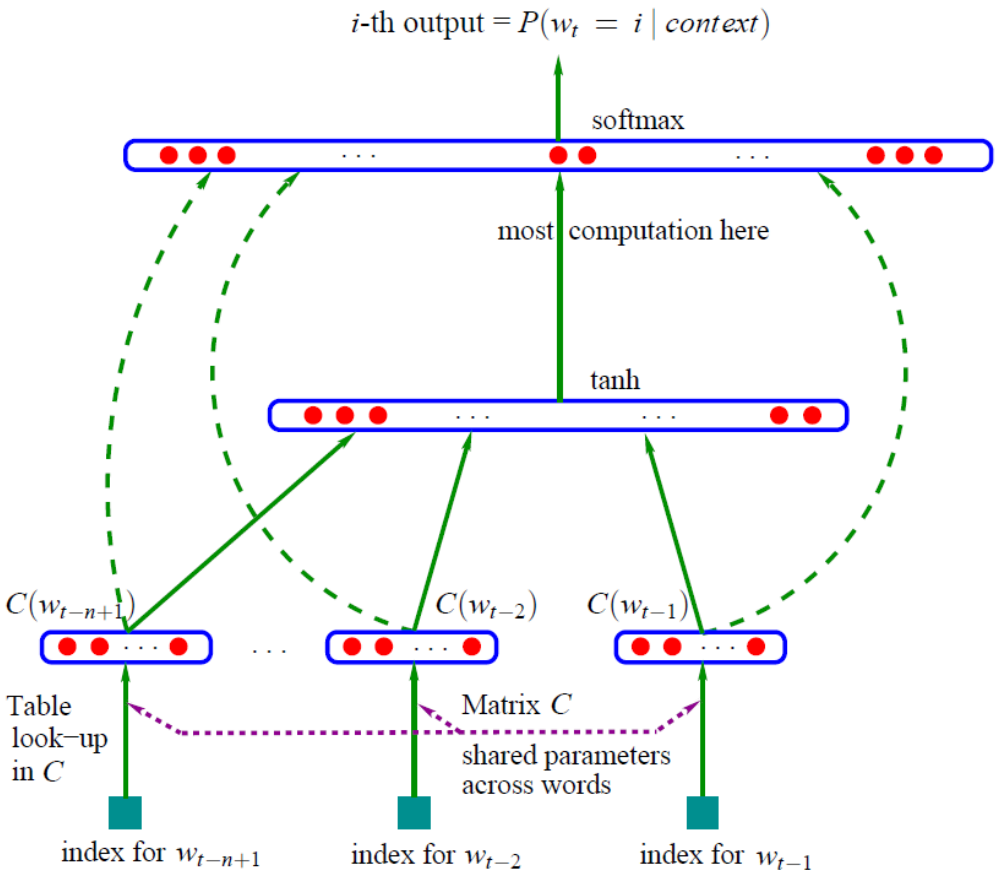
Perplexity of a distribution vs. perplexity as an evaluation metric (2)

- **Perplexity as an evaluation metric** is related to the cross-entropy via $PP_{\hat{p}}(\mathbf{w}) = 2^{H_{\hat{p}}(\mathbf{w})}$
- **Perplexity as an evaluation metric** is the perplexity of a **model distribution** with respect to a **data set** (= an empirical distribution)
- The **perplexity of a distribution** $p(X)$ is just $2^{H_p(X)}$
- (The former should ideally have been called "cross-perplexity"...)
  - What is the max of perplexity as an eval metric? (infinity)
  - What is the perplexity of a uniform model distribution evaluated on an arbitrary data set? (Still the cardinality = vocabulary size)
  - How do we compare LMs with different vocabularies? (We don't)

Neural language models (NLM)

First neural language model: Multi-layer perceptron

- Instead of smoothing, handles sparse data problem by representing each word as a continuous-valued vector (word embedding) given by matrix C



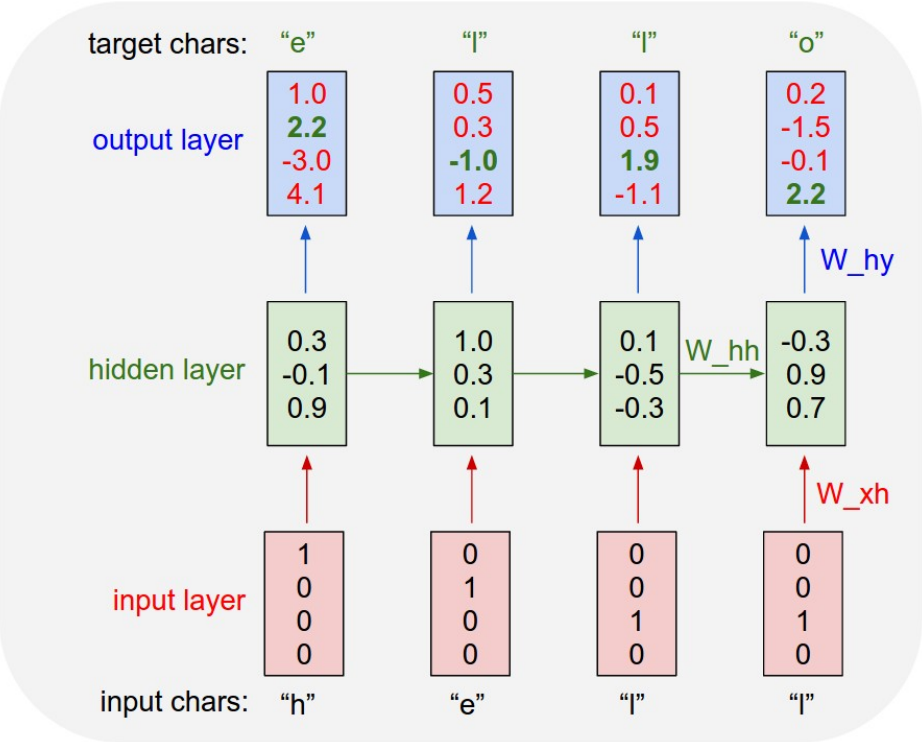
[Bengio+ 2003]

NLM perplexity results

Brown corpus: 1.1M words training texts, vocabulary size 16K

	n	c	h	m	direct	mix	train.	valid.	test.
MLP1	5		50	60	yes	no	182	284	268
MLP2	5		50	60	yes	yes		275	257
MLP3	5		0	60	yes	no	201	327	310
MLP4	5		0	60	yes	yes		286	272
MLP5	5		50	30	yes	no	209	296	279
MLP6	5		50	30	yes	yes		273	259
MLP7	3		50	30	yes	no	210	309	293
MLP8	3		50	30	yes	yes		284	270
MLP9	5		100	30	no	no	175	280	276
MLP10	5		100	30	no	yes		265	**252**
Del. Int.	3						31	352	336
Kneser-Ney back-off	3							334	323
Kneser-Ney back-off	4							332	321
Kneser-Ney back-off	5							332	321
class-based back-off	3	150						348	334
class-based back-off	3	200						354	340
class-based back-off	3	500						326	**312**
class-based back-off	3	1000						335	319
class-based back-off	3	2000						343	326
class-based back-off	4	500						327	312
class-based back-off	5	500						327	312

Basic RNN language model



$$h_t = \sigma(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$
$$y_t = \text{softmax}(W_{hy}h_t + b_y)$$

(All lower-case variables are vectors; upper-case are matrices)

Learning: maximize $\prod_t p(x_t|x_{<t})$

Another result snapshot: RNN LMs in 2016

One Billion Word corpus: 0.8B words, vocabulary size 800K

Model	Test Perplexity	Number of Params [billions]
Sigmoid-RNN-2048 (Ji et al., 2015a)	68.3	4.1
Interpolated KN 5-gram, 1.1B n-grams (Chelba et al., 2013)	67.6	1.76
Sparse Non-Negative Matrix LM (Shazeer et al., 2015)	52.9	33
RNN-1024 + MaxEnt 9-gram features (Chelba et al., 2013)	51.3	20
LSTM-512-512	54.1	0.82
LSTM-1024-512	48.2	0.82
LSTM-2048-512	43.7	0.83
LSTM-8192-2048 (No Dropout)	37.9	3.3
LSTM-8192-2048 (50% Dropout)	32.2	3.3
2-Layer LSTM-8192-1024 (BIG LSTM)	30.6	1.8
BIG LSTM+CNN Inputs	**30.0**	**1.04**
BIG LSTM+CNN Inputs + CNN Softmax	39.8	**0.29**
BIG LSTM+CNN Inputs + CNN Softmax + 128-dim correction	35.8	**0.39**
BIG LSTM+CNN Inputs + Char LSTM predictions	47.9	**0.23**

More recent language models

- Most recent work is based on transformers
- Trained on much more data, with many more parameters
- First transformer LM: GPT from OpenAI (2018)
- GPT-2 (2020): Reduced perplexity on one data set from 100 to 9
- Some stats: GPT-3 (2020)
  - 100s of billions of training text tokens (sub-words)
  - 175 billion parameters
- The LMs we use today are fine-tuned in a supervised setting
- For more on recent language model work, take NLP courses at TTIC and U. Chicago!

A few GPT perplexity results

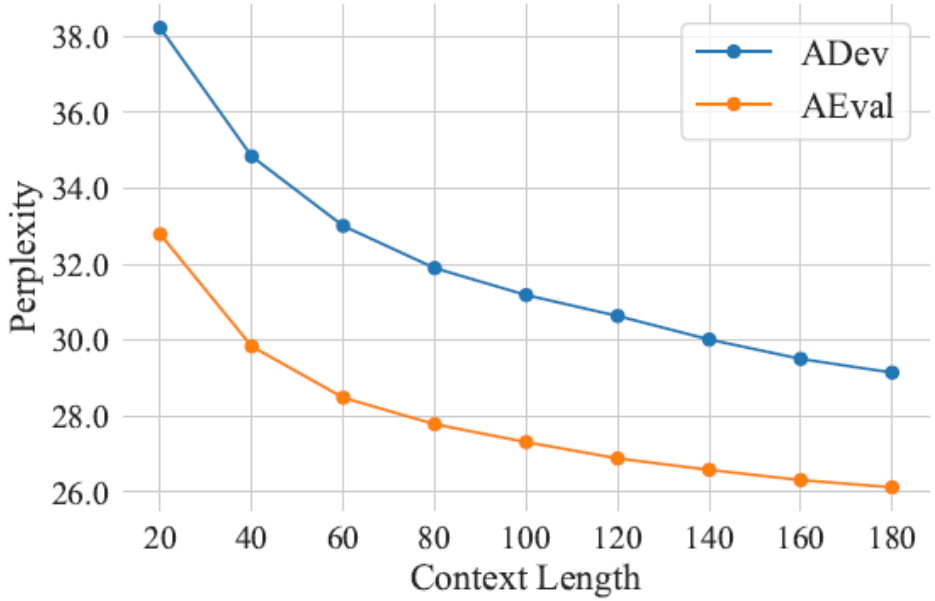


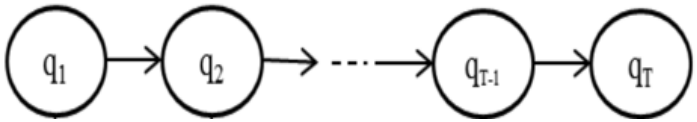
Fig. 3. Preplexity on AMI (ADev and AEval) for GPT fine-tuned with different context lengths. The context length is the number of tokens the model can see for giving an output.

[Zheng+ 2021]

Hidden Markov models

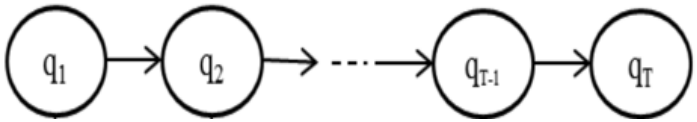
From n-gram LMs to hidden Markov models

n-gram LM as a graphical model:

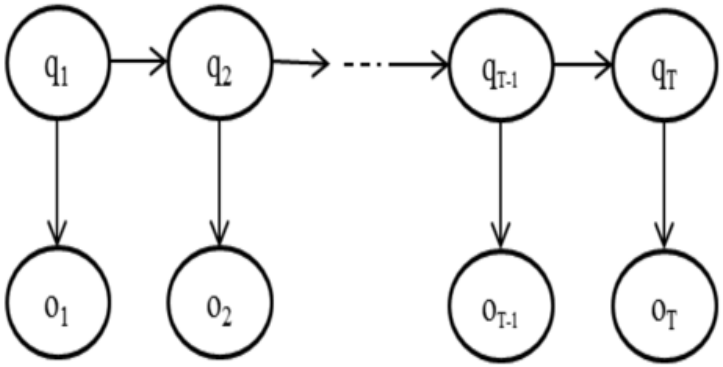


From *n*-gram LMs to hidden Markov models

n-gram LM as a graphical model:

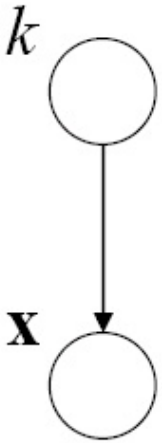


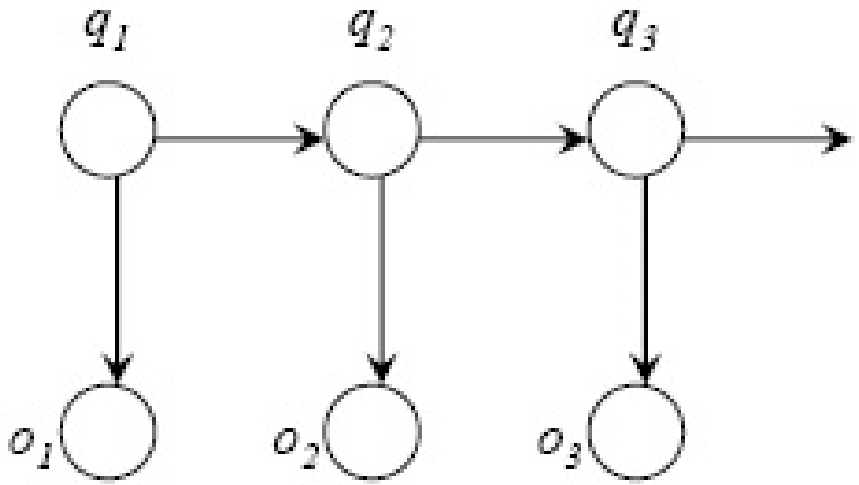
An HMM is like a language model where there is an *n*-gram over topics, and words are drawn independently given the topic:



From mixture models to HMMs

- HMMs are also an extension of mixture models, where the mixture components are not independent across data points
- Graphical model representation for mixture models and HMMs:





Hidden Markov models (HMMs): An example

- Your friend is in a distant city, and you talk with him once per day. His mood during your conversations ("words") depends on the daily weather ("topic") at his location.
- 3 weather conditions (*states*): Fair (sunny), Cloudy, Rainy.
- 2 moods (i.e. observations): Happy, Unhappy
- Example observation sequence for 4 days: $\mathbf{O} = HHUH$

Hidden Markov models (HMMs): An example

- Your friend is in a distant city, and you talk with him once per day. His mood during your conversations ("words") depends on the daily weather ("topic") at his location.
- 3 weather conditions (*states*): Fair (sunny), Cloudy, Rainy.
- 2 moods (i.e. observations): Happy, Unhappy
- Example observation sequence for 4 days: $\mathbf{O} = HHUH$
- The 3 problems:
  1. What is the probability of this sequence? (the *scoring* problem)
  2. What is your best guess of the sequence of weather states, $q_1, \ldots, q_4$? (the *decoding* problem)
  3. Given a large number of observations, how could you learn a model of the weather-mood system? (the *training* problem)

HMMs: An example (2)

Modeling the weather-mood system as a hidden Markov model:

HMMs: An example (2)

Modeling the weather-mood system as a hidden Markov model:

- On the first day, the *a priori* probabilities of the three weather states F, C, R are $\{0.4, 0.3, 0.3\}$, respectively

HMMs: An example (2)

Modeling the weather-mood system as a hidden Markov model:

- On the first day, the *a priori* probabilities of the three weather states F, C, R are $\{0.4, 0.3, 0.3\}$, respectively
- On any subsequent day t, the weather state q_t depends on the previous day's weather (and on nothing else), according to $P(q_{t+1}|q_t)$
  - I.e., the state sequence is a *Markov chain*
  - It is *hidden* since you don't observe the weather.

HMMs: An example (2)

Modeling the weather-mood system as a hidden Markov model:

- On the first day, the *a priori* probabilities of the three weather states F, C, R are $\{0.4, 0.3, 0.3\}$, respectively
- On any subsequent day t, the weather state q_t depends on the previous day's weather (and on nothing else), according to $P(q_{t+1}|q_t)$
  - I.e., the state sequence is a *Markov chain*
  - It is *hidden* since you don't observe the weather.
- Each day, your friend's mood o_t depends probabilistically on that day's weather (and on nothing else), according to $P(o_t|q_t)$

HMMs: An example (2)

Modeling the weather-mood system as a hidden Markov model:

- On the first day, the *a priori* probabilities of the three weather states F, C, R are $\{0.4, 0.3, 0.3\}$, respectively
- On any subsequent day t, the weather state q_t depends on the previous day's weather (and on nothing else), according to $P(q_{t+1}|q_t)$
  - I.e., the state sequence is a *Markov chain*
  - It is *hidden* since you don't observe the weather.
- Each day, your friend's mood o_t depends probabilistically on that day's weather (and on nothing else), according to $P(o_t|q_t)$

$P(o_t \mid q_t)$	$o_t = H$	$o_t = U$
$q_t = F$	0.9	0.1
$q_t = C$	0.5	0.5
$q_t = R$	0.2	0.8

$P(q_{t+1} \mid q_t)$	$q_{t+1} = F$	$q_{t+1} = C$	$q_{t+1} = R$
$q_t = F$	0.8	0.2	0
$q_t = C$	0.3	0.4	0.3
$q_t = R$	0	0.3	0.7

Elements of the weather-mood HMM

- States: $N = 3$; let state 1 be F, state 2 be C, state 3 be R
- Obervation labels: $M = 2$; let $v_1 = H, v_2 = U$
- Model parameters $\lambda = \{\mathbf{A}, \mathbf{B}, \pi\}$:

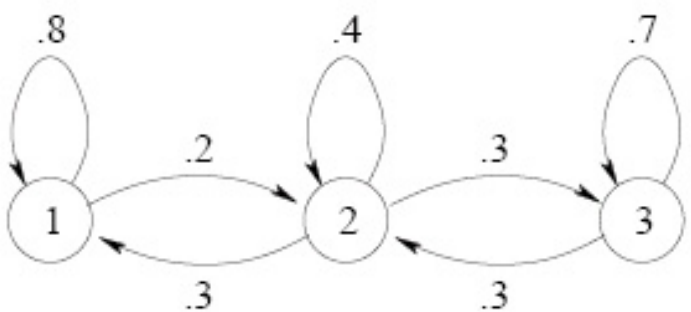
$$\pi = [0.4\ 0.3\ 0.3], \quad A = \begin{bmatrix} 0.8 & 0.2 & 0 \\ 0.3 & 0.4 & 0.3 \\ 0 & 0.3 & 0.7 \end{bmatrix}, \quad B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

Elements of the weather-mood HMM

- States: $N = 3$; let state 1 be F, state 2 be C, state 3 be R
- Obervation labels: $M = 2$; let $v_1 = H, v_2 = U$
- Model parameters $\lambda = \{\mathbf{A}, \mathbf{B}, \pi\}$:

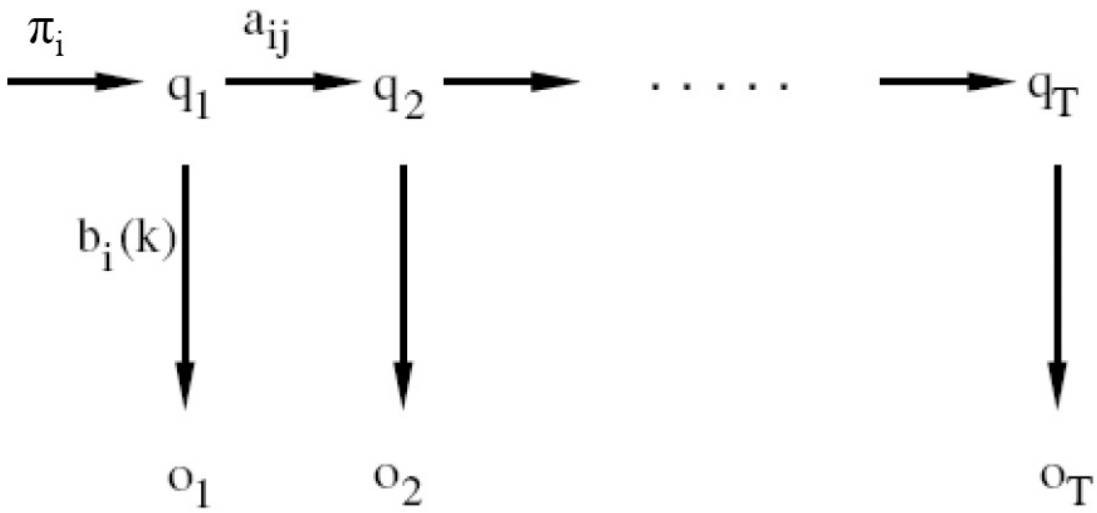
$$\pi = [0.4\ 0.3\ 0.3], \quad A = \begin{bmatrix} 0.8 & 0.2 & 0 \\ 0.3 & 0.4 & 0.3 \\ 0 & 0.3 & 0.7 \end{bmatrix}, \quad B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

- $\mathbf{A}$ can also be represented via a *state transition diagram*:



Generation of observations from an HMM

1. Draw an initial state $q_1 = i$ from the initial distribution π
2. For $t = 1$ to T:
   - Choose $o_t = v_k$ according to state i's observation distribution $b_i(k)$
   - Choose a new state $q_{t+1} = j$ according to state i's transition probabilities a_{ij}. Let $i \leftarrow j$



The three basic HMM problems

The three basic HMM problems

1. *Scoring* (inference type 1): Given an observation sequence $\mathbf{O} = \{\mathbf{o}_1, \ldots, \mathbf{o}_T\}$ and a model $\lambda = \{\mathbf{A}, \mathbf{B}, \pi\}$, how do we compute $P(\mathbf{O}|\lambda)$, the probability of the observation sequence?
   $\longrightarrow$ The forward & backward algorithms

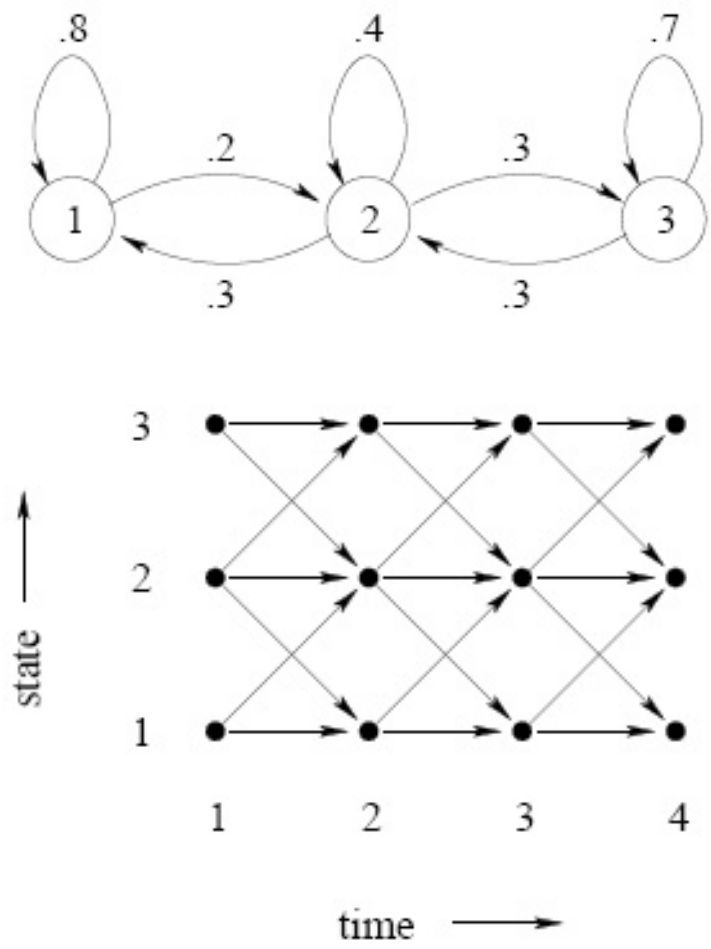
The three basic HMM problems

1. *Scoring* (inference type 1): Given an observation sequence $\mathbf{O} = \{\mathbf{o}_1, \ldots, \mathbf{o}_T\}$ and a model $\lambda = \{\mathbf{A}, \mathbf{B}, \pi\}$, how do we compute $P(\mathbf{O}|\lambda)$, the probability of the observation sequence?
   $\longrightarrow$ The forward & backward algorithms
2. *Decoding* (inference type 2): Given an observation sequence $\mathbf{O} = \{o_1, \ldots, o_T\}$, how do we choose the state sequence $\mathbf{q} = \{q_1, \ldots, q_T\}$ most likely to have generated the observations?
   $\longrightarrow$ The Viterbi algorithm

The three basic HMM problems

1. *Scoring* (inference type 1): Given an observation sequence $\mathbf{O} = \{\mathbf{o}_1, \ldots, \mathbf{o}_T\}$ and a model $\lambda = \{\mathbf{A}, \mathbf{B}, \pi\}$, how do we compute $P(\mathbf{O}|\lambda)$, the probability of the observation sequence?
   $\longrightarrow$ The forward & backward algorithms
2. *Decoding* (inference type 2): Given an observation sequence $\mathbf{O} = \{o_1, \ldots, o_T\}$, how do we choose the state sequence $\mathbf{q} = \{q_1, \ldots, q_T\}$ most likely to have generated the observations?
   $\longrightarrow$ The Viterbi algorithm
3. *Training* (learning): Given a training set of observations $\mathbf{O}$, how do we set the model parameters $\lambda = \{\mathbf{A}, \mathbf{B}, \pi\}$ to maximize $P(\mathbf{O}|\lambda)$ (maximum-likelihood estimation)
   $\longrightarrow$ The Baum-Welch algorithm (EM applied to HMMs)

Representing transition diagram with a trellis



This representation is useful in visualizing the algorithms involved in solving the three problems

Problem 1 (inference): Computing $p(\mathbf{O}|\lambda)$

$$
\begin{aligned}
p(\mathbf{O}|\lambda) &= \sum_{\text{all } \mathbf{q}} p(\mathbf{O}, \mathbf{q}|\lambda) \\
p(\mathbf{O}, \mathbf{q}|\lambda) &= p(\mathbf{O}|\mathbf{q}, \lambda) p(\mathbf{q}|\lambda)
\end{aligned}
$$

Problem 1 (inference): Computing $p(\mathbf{O}|\lambda)$

$$
\begin{aligned}
p(\mathbf{O}|\lambda) &= \sum_{\text{all } \mathbf{q}} p(\mathbf{O}, \mathbf{q}|\lambda) \\
p(\mathbf{O}, \mathbf{q}|\lambda) &= p(\mathbf{O}|\mathbf{q}, \lambda) p(\mathbf{q}|\lambda)
\end{aligned}
$$

- Consider some *fixed* state sequence $\mathbf{q} = q_1 q_2 \dots q_T$. Then:

$$
\begin{aligned}
p(\mathbf{O}|\mathbf{q}, \lambda) &= b_{q_1}(\mathbf{o}_1) b_{q_2}(\mathbf{o}_2) \dots b_{q_T}(\mathbf{o}_T) \\
p(\mathbf{q}|\lambda) &= \pi_{q_1} a_{q_1 q_2} a_{q_2 q_3} \dots a_{q_{T-1} q_T}
\end{aligned}
$$

Problem 1 (inference): Computing $p(\mathbf{O}|\lambda)$

$$
\begin{aligned}
p(\mathbf{O}|\lambda) &= \sum_{\text{all } \mathbf{q}} p(\mathbf{O}, \mathbf{q}|\lambda) \\
p(\mathbf{O}, \mathbf{q}|\lambda) &= p(\mathbf{O}|\mathbf{q}, \lambda) p(\mathbf{q}|\lambda)
\end{aligned}
$$

- Consider some *fixed* state sequence $\mathbf{q} = q_1 q_2 \dots q_T$. Then:

$$
\begin{aligned}
p(\mathbf{O}|\mathbf{q}, \lambda) &= b_{q_1}(\mathbf{o}_1) b_{q_2}(\mathbf{o}_2) \dots b_{q_T}(\mathbf{o}_T) \\
p(\mathbf{q}|\lambda) &= \pi_{q_1} a_{q_1 q_2} a_{q_2 q_3} \dots a_{q_{T-1} q_T}
\end{aligned}
$$

- Therefore,

$$
p(\mathbf{O}|\lambda) = \sum_{q_1, q_2, \dots, q_T} \pi_{q_1} b_{q_1}(\mathbf{o}_1) a_{q_1 q_2} b_{q_2}(\mathbf{o}_2) \dots a_{q_{T-1} q_T} b_{q_T}(\mathbf{o}_T)
$$

Problem 1 (inference): Computing $p(\mathbf{O}|\lambda)$

$$
\begin{aligned}
p(\mathbf{O}|\lambda) &= \sum_{\text{all } \mathbf{q}} p(\mathbf{O}, \mathbf{q}|\lambda) \\
p(\mathbf{O}, \mathbf{q}|\lambda) &= p(\mathbf{O}|\mathbf{q}, \lambda) p(\mathbf{q}|\lambda)
\end{aligned}
$$

- Consider some *fixed* state sequence $\mathbf{q} = q_1 q_2 \dots q_T$. Then:

$$
\begin{aligned}
p(\mathbf{O}|\mathbf{q}, \lambda) &= b_{q_1}(\mathbf{o}_1) b_{q_2}(\mathbf{o}_2) \dots b_{q_T}(\mathbf{o}_T) \\
p(\mathbf{q}|\lambda) &= \pi_{q_1} a_{q_1 q_2} a_{q_2 q_3} \dots a_{q_{T-1} q_T}
\end{aligned}
$$

- Therefore,
$$
p(\mathbf{O}|\lambda) = \sum_{q_1, q_2, \dots, q_T} \pi_{q_1} b_{q_1}(\mathbf{o}_1) a_{q_1 q_2} b_{q_2}(\mathbf{o}_2) \dots a_{q_{T-1} q_T} b_{q_T}(\mathbf{o}_T)
$$

- N^T possible sequences $\mathbf{q} \Rightarrow$ computation $\approx 2T \cdot N^T$. For $N = 5, T = 100$, this is $2 \cdot 100 \cdot 5^{100} \approx 10^{72}$ computations!

Computing $p(O|\lambda)$: The forward algorithm

- A dynamic programming algorithm

Computing $p(O|\lambda)$: The forward algorithm

- A dynamic programming algorithm
- Define the "forward" variable $\alpha_t(i)$:
  $\alpha_t(i) = P(\mathbf{o}_1 \mathbf{o}_2 \dots \mathbf{o}_t, q_t = i|\lambda), \quad 1 \leq t \leq T, 1 \leq i \leq N$

Computing $p(O|\lambda)$: The forward algorithm

- A dynamic programming algorithm
- Define the "forward" variable $\alpha_t(i)$:
  $\alpha_t(i) = P(\mathbf{o}_1 \mathbf{o}_2 \dots \mathbf{o}_t, q_t = i|\lambda), \quad 1 \leq t \leq T, 1 \leq i \leq N$
- For $t = 1$: $\alpha_1(i) = \pi_i b_i(\mathbf{o}_1), 1 \leq i \leq N$

Computing $p(O|\lambda)$: The forward algorithm

- A dynamic programming algorithm
- Define the "forward" variable $\alpha_t(i)$:
  $\alpha_t(i) = P(\mathbf{o}_1 \mathbf{o}_2 \dots \mathbf{o}_t, q_t = i|\lambda), \quad 1 \leq t \leq T, 1 \leq i \leq N$
- For $t = 1$: $\alpha_1(i) = \pi_i b_i(\mathbf{o}_1), 1 \leq i \leq N$
- For $t > 1$: Sum over all ways of getting to current state at time t:
  $$\alpha_t(j) = \left[\sum_{i=1}^{N} \alpha_{t-1}(i) a_{ij} \right] b_j(\mathbf{o}_t)$$

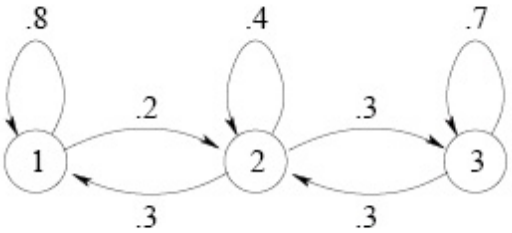
Computing $p(O|\lambda)$: The forward algorithm

- A dynamic programming algorithm
- Define the "forward" variable $\alpha_t(i)$:
  $\alpha_t(i) = P(\mathbf{o}_1 \mathbf{o}_2 \ldots \mathbf{o}_t, q_t = i|\lambda), \quad 1 \leq t \leq T, 1 \leq i \leq N$
- For $t = 1$: $\alpha_1(i) = \pi_i b_i(\mathbf{o}_1), 1 \leq i \leq N$
- For $t > 1$: Sum over all ways of getting to current state at time t:
  $$\alpha_t(j) = \left[\sum_{i=1}^{N} \alpha_{t-1}(i) a_{ij} \right] b_j(\mathbf{o}_t)$$
- Finally: $p(\mathbf{O}|\lambda) = \sum_{i=1}^{N} \alpha_T(i)$

Computing $p(O|\lambda)$: The forward algorithm

- A dynamic programming algorithm
- Define the "forward" variable $\alpha_t(i)$:
  $\alpha_t(i) = P(\mathbf{o}_1 \mathbf{o}_2 \ldots \mathbf{o}_t, q_t = i|\lambda), \quad 1 \leq t \leq T, 1 \leq i \leq N$
- For $t = 1$: $\alpha_1(i) = \pi_i b_i(\mathbf{o}_1), 1 \leq i \leq N$
- For $t > 1$: Sum over all ways of getting to current state at time t:
  $$\alpha_t(j) = \left[\sum_{i=1}^{N} \alpha_{t-1}(i) a_{ij} \right] b_j(\mathbf{o}_t)$$
- Finally: $p(\mathbf{O}|\lambda) = \sum_{i=1}^{N} \alpha_T(i)$
- Calculation is on the order of $N^2 T$. $N = 5, T = 100 \Rightarrow \sim 2500$ computations, instead of 10^{72}

The forward algorithm: Example



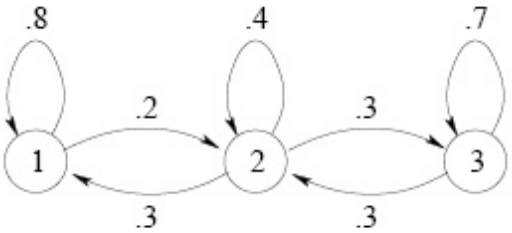
$\pi = [0.4\ 0.3\ 0.3]$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$O = HHUH$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)				
2 (C)				
1 (F)				
	H	H	U	H

The forward algorithm: Example



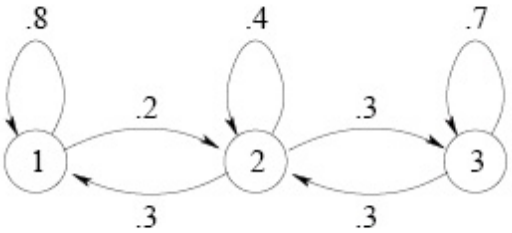
$\pi = [0.4\ 0.3\ 0.3]$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$O = HHUH$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)				
2 (C)				
1 (F)	.4			
	H	H	U	H

The forward algorithm: Example



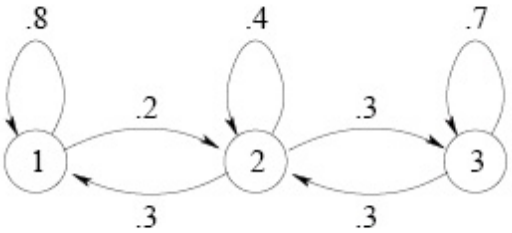
$\pi = [0.4\ 0.3\ 0.3]$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$O = HHUH$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)				
2 (C)				
1 (F)	$.4\times.9$			
	H	H	U	H

The forward algorithm: Example



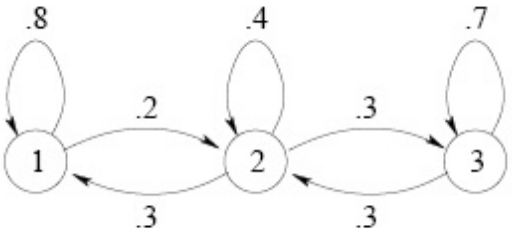
$\pi = [0.4\ 0.3\ 0.3]$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$O = HHUH$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)				
2 (C)				
1 (F)	.4×.9 = .36			
	H	H	U	H

The forward algorithm: Example



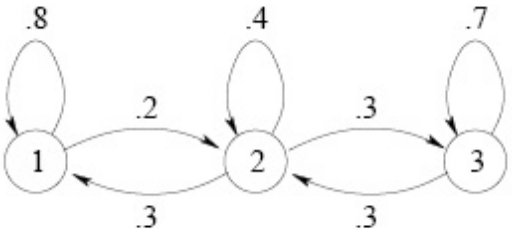
$\pi = [0.4\ 0.3\ 0.3]$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$O = HHUH$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)				
2 (C)	$.3 \times .5 = .15$			
1 (F)	$.4 \times .9 = .36$			
	H	H	U	H

The forward algorithm: Example



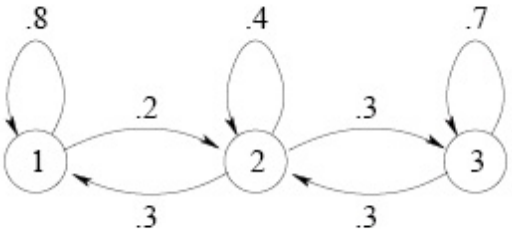
$\pi = [0.4\ 0.3\ 0.3]$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$O = HHUH$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)	$.3 \times .2 = .06$			
2 (C)	$.3 \times .5 = .15$			
1 (F)	$.4 \times .9 = .36$			
	H	H	U	H

The forward algorithm: Example



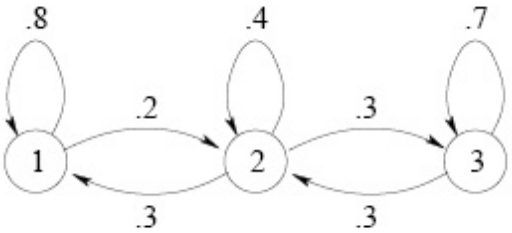
$\pi = [0.4\ 0.3\ 0.3]$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$O = HHUH$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)	$.3 \times .2 = .06$			
2 (C)	$.3 \times .5 = .15$			
1 (F)	$.4 \times .9 = .36$	$(.36 \times .8$		
	H	H	U	H

The forward algorithm: Example



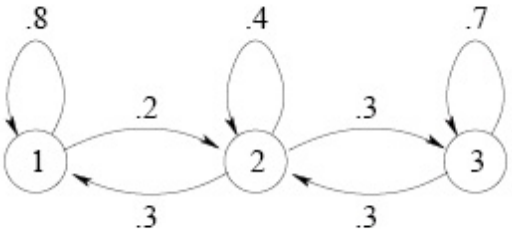
$$\pi = [0.4\ 0.3\ 0.3]$$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$$O = HHUH$$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)	$.3 \times .2 = .06$			
2 (C)	$.3 \times .5 = .15$			
1 (F)	$.4 \times .9 = .36$	$(.36 \times .8 + .15 \times .3)$		
	H	H	U	H

The forward algorithm: Example



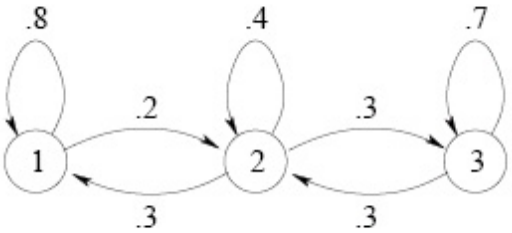
$$\pi = [0.4\ 0.3\ 0.3]$$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$$O = HHUH$$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)	$.3 \times .2 = .06$			
2 (C)	$.3 \times .5 = .15$			
1 (F)	$.4 \times .9 = .36$	$(.36 \times .8 + .15 \times .3) \times .9$		
	H	H	U	H

The forward algorithm: Example



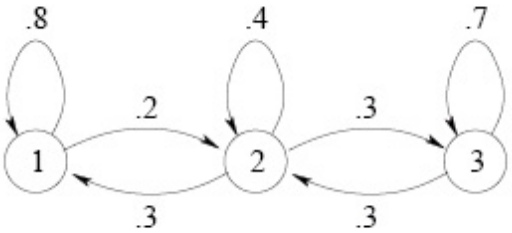
$$\pi = [0.4\ 0.3\ 0.3]$$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$$O = HHUH$$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)	$.3 \times .2 = .06$			
2 (C)	$.3 \times .5 = .15$			
1 (F)	$.4 \times .9 = .36$	$(.36 \times .8 + .15 \times .3) \times .9 = .2997$		
	H	H	U	H

The forward algorithm: Example



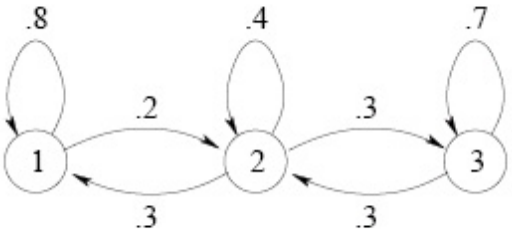
$$\pi = [0.4\ 0.3\ 0.3]$$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$$O = HHUH$$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)	$.3 \times .2 = .06$			
2 (C)	$.3 \times .5 = .15$	$(.36 \times .2$		
1 (F)	$.4 \times .9 = .36$	$(.36 \times .8 + .15 \times .3) \times .9 = .2997$		
	H	H	U	H

The forward algorithm: Example



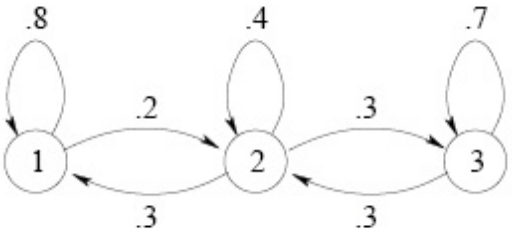
$\pi = [0.4\ 0.3\ 0.3]$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$O = HHUH$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)	$.3 \times .2 = .06$			
2 (C)	$.3 \times .5 = .15$	$(.36 \times .2 + .15 \times .4$		
1 (F)	$.4 \times .9 = .36$	$(.36 \times .8 + .15 \times .3) \times .9 = .2997$		
	H	H	U	H

The forward algorithm: Example



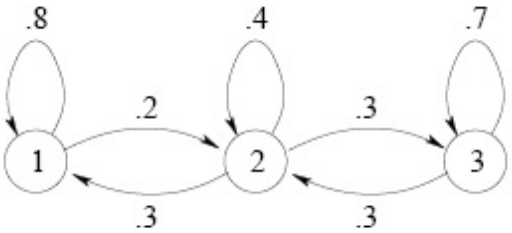
$\pi = [0.4\ 0.3\ 0.3]$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$O = HHUH$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (*R*)	$.3 \times .2 = .06$			
2 (*C*)	$.3 \times .5 = .15$	$(.36 \times .2 + .15 \times .4 + .06 \times .3)$		
1 (*F*)	$.4 \times .9 = .36$	$(.36 \times .8 + .15 \times .3) \times .9 = .2997$		
	H	*H*	*U*	*H*

The forward algorithm: Example



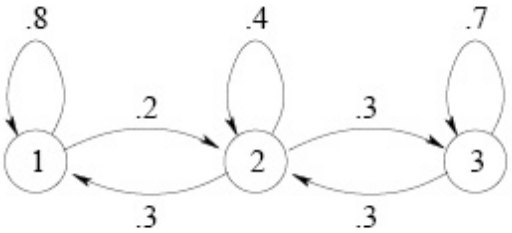
$$\pi = [0.4\ 0.3\ 0.3]$$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$$O = HHUH$$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)	$.3 \times .2 = .06$			
2 (C)	$.3 \times .5 = .15$	$(.36 \times .2 + .15 \times .4 + .06 \times .3) \times .5$		
1 (F)	$.4 \times .9 = .36$	$(.36 \times .8 + .15 \times .3) \times .9 = .2997$		
	H	H	U	H

The forward algorithm: Example



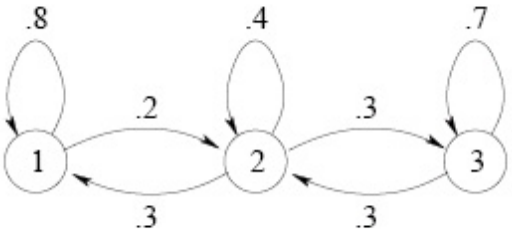
$$\pi = [0.4\ 0.3\ 0.3]$$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$$O = HHUH$$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)	$.3 \times .2 = .06$			
2 (C)	$.3 \times .5 = .15$	$(.36 \times .2 + .15 \times .4 + .06 \times .3) \times .5 = .075$		
1 (F)	$.4 \times .9 = .36$	$(.36 \times .8 + .15 \times .3) \times .9 = .2997$		
	H	H	U	H

The forward algorithm: Example



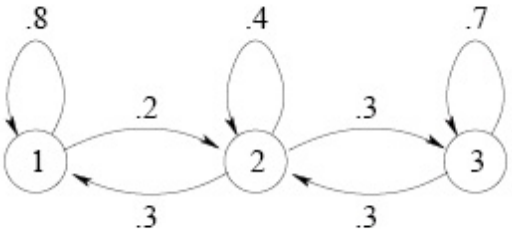
$$\pi = [0.4\ 0.3\ 0.3]$$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$$O = HHUH$$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)	$.3 \times .2 = .06$	$(.15 \times .3 + .06 \times .7)$		
2 (C)	$.3 \times .5 = .15$	$(.36 \times .2 + .15 \times .4 + .06 \times .3) \times .5 = .075$		
1 (F)	$.4 \times .9 = .36$	$(.36 \times .8 + .15 \times .3) \times .9 = .2997$		
	H	H	U	H

The forward algorithm: Example



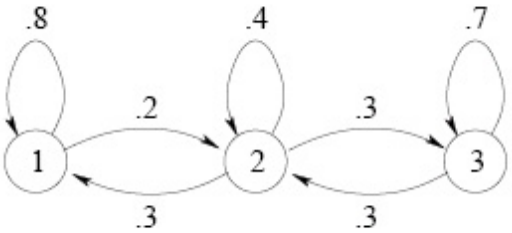
$$\pi = [0.4\ 0.3\ 0.3]$$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$$O = HHUH$$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)	$.3 \times .2 = .06$	$(.15 \times .3 + .06 \times .7) \times .2$		
2 (C)	$.3 \times .5 = .15$	$(.36 \times .2 + .15 \times .4 + .06 \times .3) \times .5 = .075$		
1 (F)	$.4 \times .9 = .36$	$(.36 \times .8 + .15 \times .3) \times .9 = .2997$		
	H	H	U	H

The forward algorithm: Example



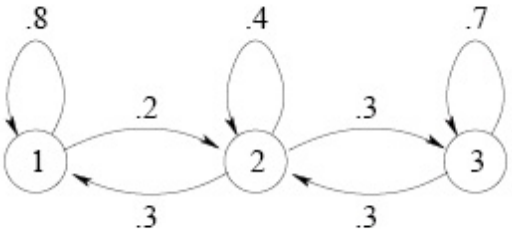
$$\pi = [0.4\ 0.3\ 0.3]$$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$$O = HHUH$$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)	$.3 \times .2 = .06$	$(.15 \times .3 + .06 \times .7) \times .2 = .0174$		
2 (C)	$.3 \times .5 = .15$	$(.36 \times .2 + .15 \times .4 + .06 \times .3) \times .5 = .075$		
1 (F)	$.4 \times .9 = .36$	$(.36 \times .8 + .15 \times .3) \times .9 = .2997$		
	H	H	U	H

The forward algorithm: Example



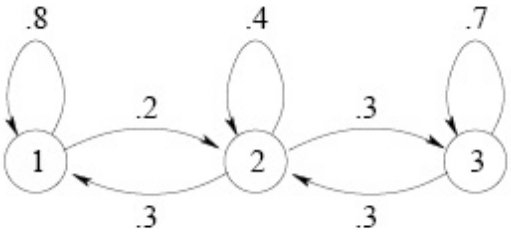
$$\pi = [0.4\ 0.3\ 0.3]$$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$$O = HHUH$$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)	$.3 \times .2 = .06$	$(.15 \times .3 + .06 \times .7) \times .2 = .0174$		
2 (C)	$.3 \times .5 = .15$	$(.36 \times .2 + .15 \times .4 + .06 \times .3) \times .5 = .075$		
1 (F)	$.4 \times .9 = .36$	$(.36 \times .8 + .15 \times .3) \times .9 = .2997$	$(.2997 \times .8 + .075 \times .3) \times .1 = .026226$	
	H	H	U	H

The forward algorithm: Example



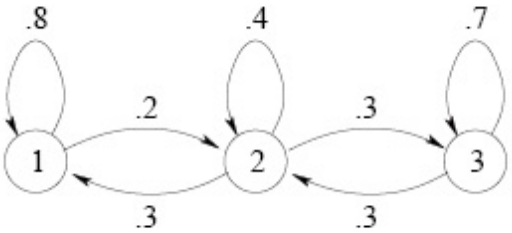
$$\pi = [0.4\ 0.3\ 0.3]$$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$$O = HHUH$$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)	$.3 \times .2$ $= .06$	$(.15 \times .3$ $+.06 \times .7) \times .2$ $= .0174$		
2 (C)	$.3 \times .5$ $= .15$	$(.36 \times .2$ $+.15 \times .4$ $+.06 \times .3) \times .5$ $= .075$	$(.2997 \times .2$ $+.075 \times .4$ $+.0174 \times .3) \times .5$ $= .04758$	
1 (F)	$.4 \times .9$ $= .36$	$(.36 \times .8$ $+.15 \times .3) \times .9$ $= .2997$	$(.2997 \times .8$ $+.075 \times .3) \times .1$ $= .026226$	
	H	H	U	H

The forward algorithm: Example



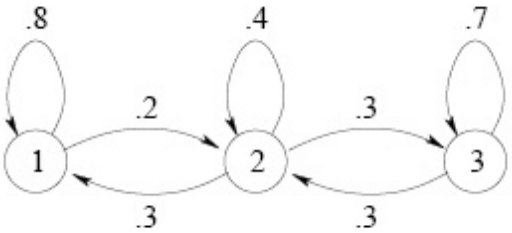
$$\pi = [0.4\ 0.3\ 0.3]$$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$$O = HHUH$$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)	$.3 \times .2$ $= .06$	$(.15 \times .3$ $+.06 \times .7) \times .2$ $= .0174$	$(.075 \times .3$ $+.0174 \times .7) \times .8$ $= .027744$	
2 (C)	$.3 \times .5$ $= .15$	$(.36 \times .2$ $+.15 \times .4$ $+.06 \times .3) \times .5$ $= .075$	$(.2997 \times .2$ $+.075 \times .4$ $+.0174 \times .3) \times .5$ $= .04758$	
1 (F)	$.4 \times .9$ $= .36$	$(.36 \times .8$ $+.15 \times .3) \times .9$ $= .2997$	$(.2997 \times .8$ $+.075 \times .3) \times .1$ $= .026226$	
	H	H	U	H

The forward algorithm: Example



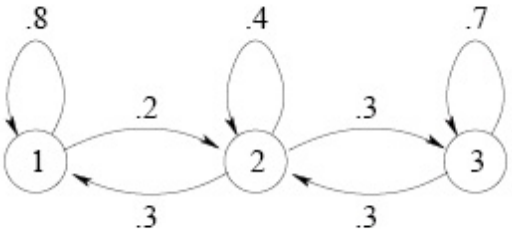
$$\pi = [0.4\ 0.3\ 0.3]$$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$$O = HHUH$$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)	$.3 \times .2$ $= .06$	$(.15 \times .3$ $+.06 \times .7) \times .2$ $= .0174$	$(.075 \times .3$ $+.0174 \times .7) \times .8$ $= .027744$	
2 (C)	$.3 \times .5$ $= .15$	$(.36 \times .2$ $+.15 \times .4$ $+.06 \times .3) \times .5$ $= .075$	$(.2997 \times .2$ $+.075 \times .4$ $+.0174 \times .3) \times .5$ $= .04758$	
1 (F)	$.4 \times .9$ $= .36$	$(.36 \times .8$ $+.15 \times .3) \times .9$ $= .2997$	$(.2997 \times .8$ $+.075 \times .3) \times .1$ $= .026226$	$\ldots$ $\ldots$ $= .03172932$
	H	H	U	H

The forward algorithm: Example



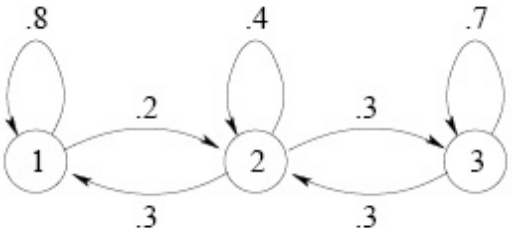
$$\pi = [0.4\ 0.3\ 0.3]$$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$$O = HHUH$$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)	$.3 \times .2$ $= .06$	$(.15 \times .3$ $+.06 \times .7) \times .2$ $= .0174$	$(.075 \times .3$ $+.0174 \times .7) \times .8$ $= .027744$	
2 (C)	$.3 \times .5$ $= .15$	$(.36 \times .2$ $+.15 \times .4$ $+.06 \times .3) \times .5$ $= .075$	$(.2997 \times .2$ $+.075 \times .4$ $+.0174 \times .3) \times .5$ $= .04758$	$\ldots$ $\ldots$ $\ldots$ $= .0163002$
1 (F)	$.4 \times .9$ $= .36$	$(.36 \times .8$ $+.15 \times .3) \times .9$ $= .2997$	$(.2997 \times .8$ $+.075 \times .3) \times .1$ $= .026226$	$\ldots$ $\ldots$ $= .03172932$
	H	H	U	H

The forward algorithm: Example



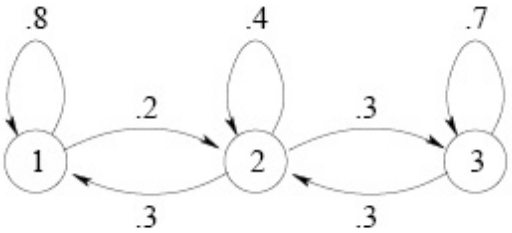
$$\pi = [0.4\ 0.3\ 0.3]$$

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$$O = HHUH$$

i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)	$.3 \times .2$ $= .06$	$(.15 \times .3$ $+.06 \times .7) \times .2$ $= .0174$	$(.075 \times .3$ $+.0174 \times .7) \times .8$ $= .027744$	$\ldots$ $\ldots$ $= .00673896$
2 (C)	$.3 \times .5$ $= .15$	$(.36 \times .2$ $+.15 \times .4$ $+.06 \times .3) \times .5$ $= .075$	$(.2997 \times .2$ $+.075 \times .4$ $+.0174 \times .3) \times .5$ $= .04758$	$\ldots$ $\ldots$ $\ldots$ $= .0163002$
1 (F)	$.4 \times .9$ $= .36$	$(.36 \times .8$ $+.15 \times .3) \times .9$ $= .2997$	$(.2997 \times .8$ $+.075 \times .3) \times .1$ $= .026226$	$\ldots$ $\ldots$ $= .03172932$
	H	H	U	H

The forward algorithm: Example



$$\pi = [0.4\ 0.3\ 0.3]$$

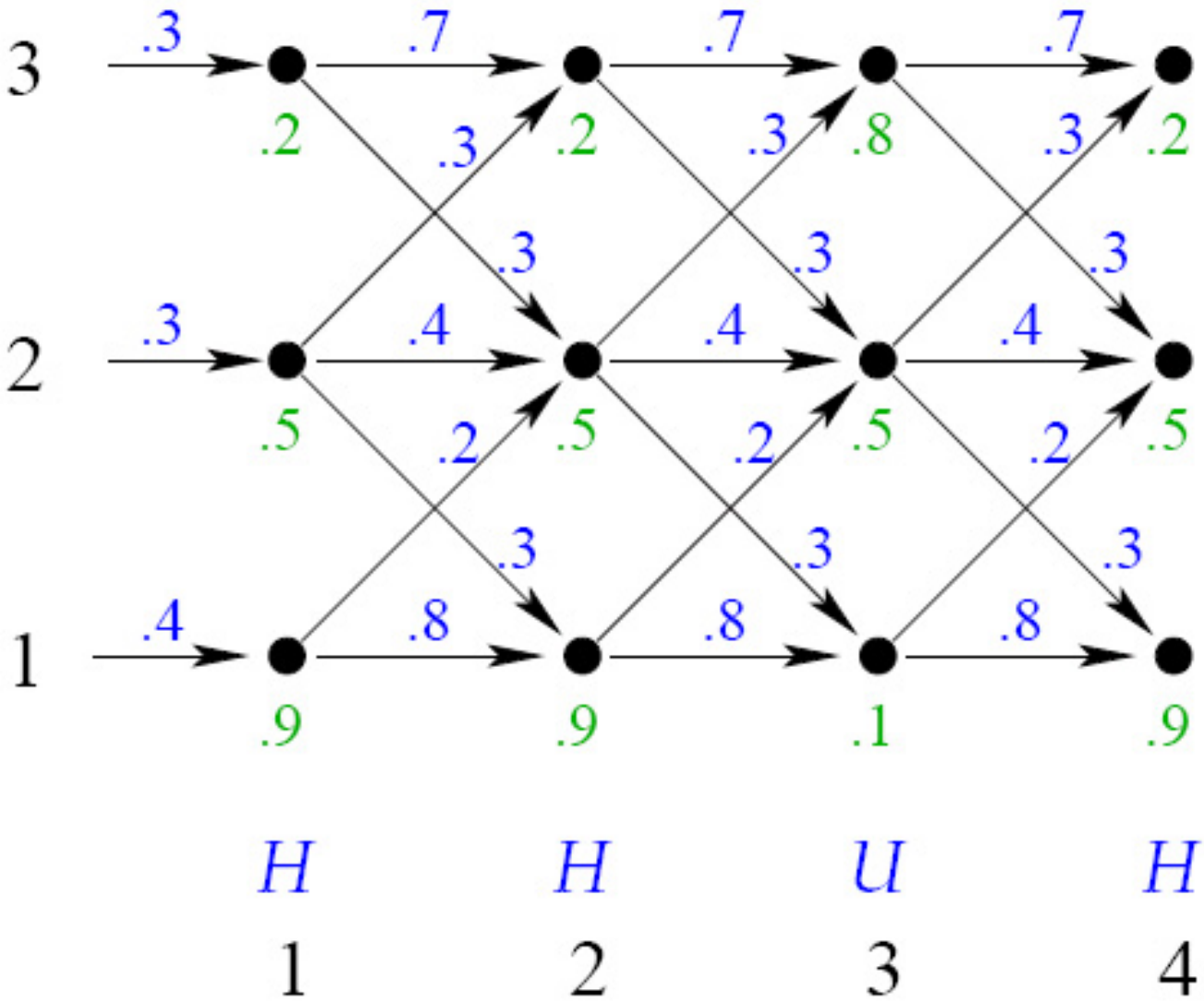
$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}$$

$$O = HHUH$$

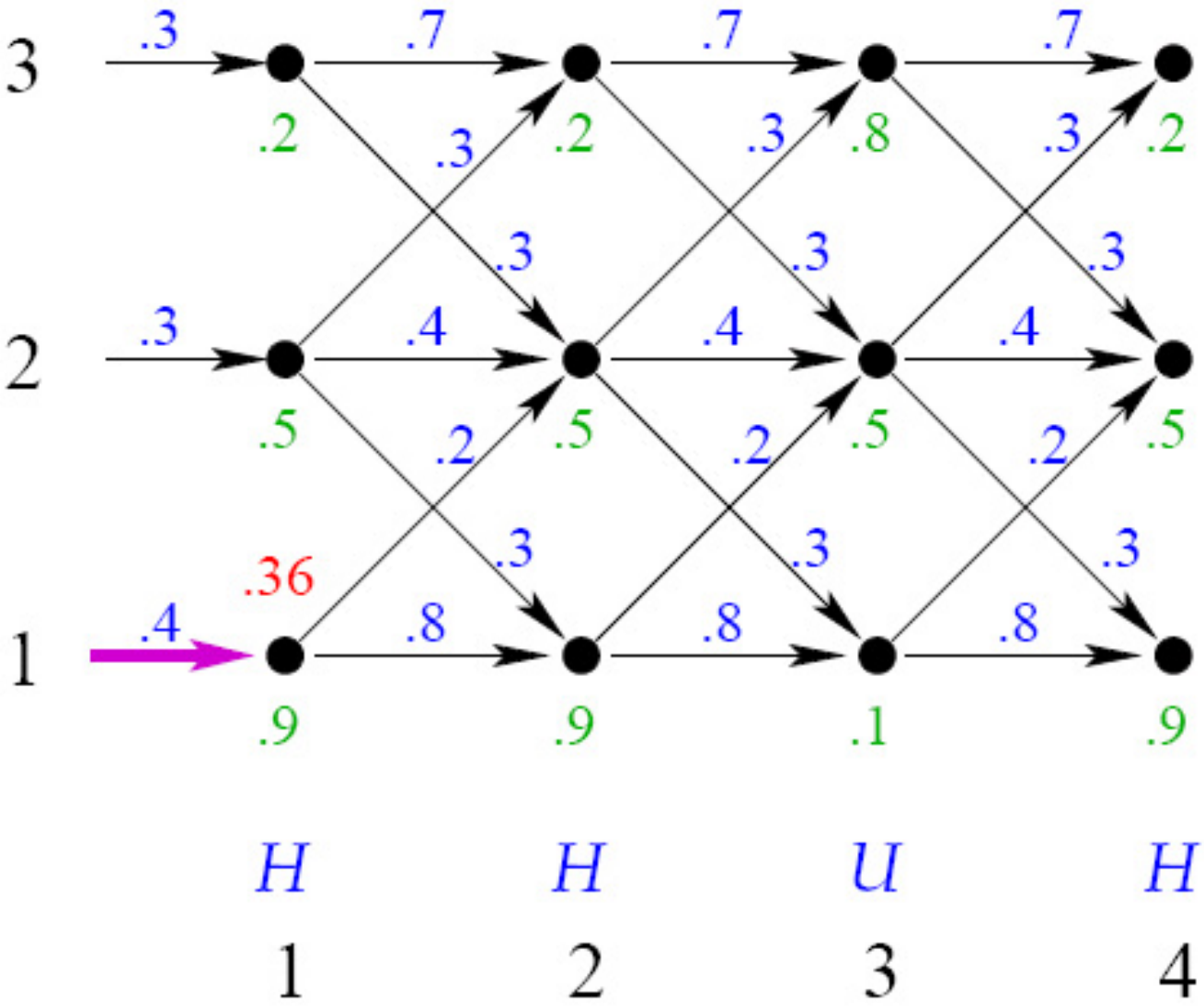
i	$\alpha_1(i)$	$\alpha_2(i)$	$\alpha_3(i)$	$\alpha_4(i)$
3 (R)	$.3 \times .2$ $= .06$	$(.15 \times .3$ $+.06 \times .7) \times .2$ $= .0174$	$(.075 \times .3$ $+.0174 \times .7) \times .8$ $= .027744$	$\ldots$ $\ldots$ $= .00673896$
2 (C)	$.3 \times .5$ $= .15$	$(.36 \times .2$ $+.15 \times .4$ $+.06 \times .3) \times .5$ $= .075$	$(.2997 \times .2$ $+.075 \times .4$ $+.0174 \times .3) \times .5$ $= .04758$	$\ldots$ $\ldots$ $\ldots$ $= .0163002$
1 (F)	$.4 \times .9$ $= .36$	$(.36 \times .8$ $+.15 \times .3) \times .9$ $= .2997$	$(.2997 \times .8$ $+.075 \times .3) \times .1$ $= .026226$	$\ldots$ $\ldots$ $= .03172932$
	H	H	U	H

Finally: $p(HHUH|\lambda) = .00673896 + .0163002 + .03172932 = .05476848$

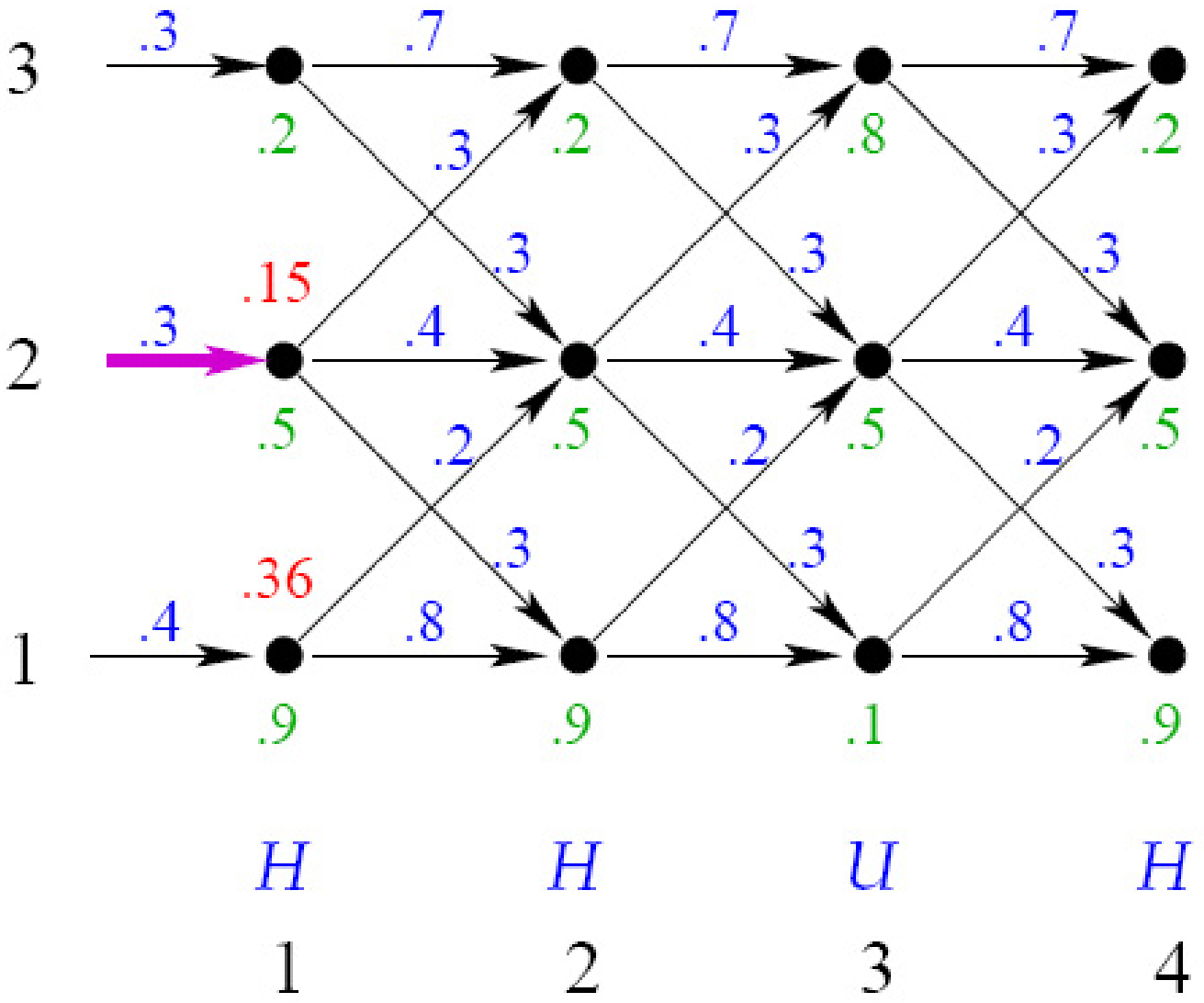
The forward algorithm: Example



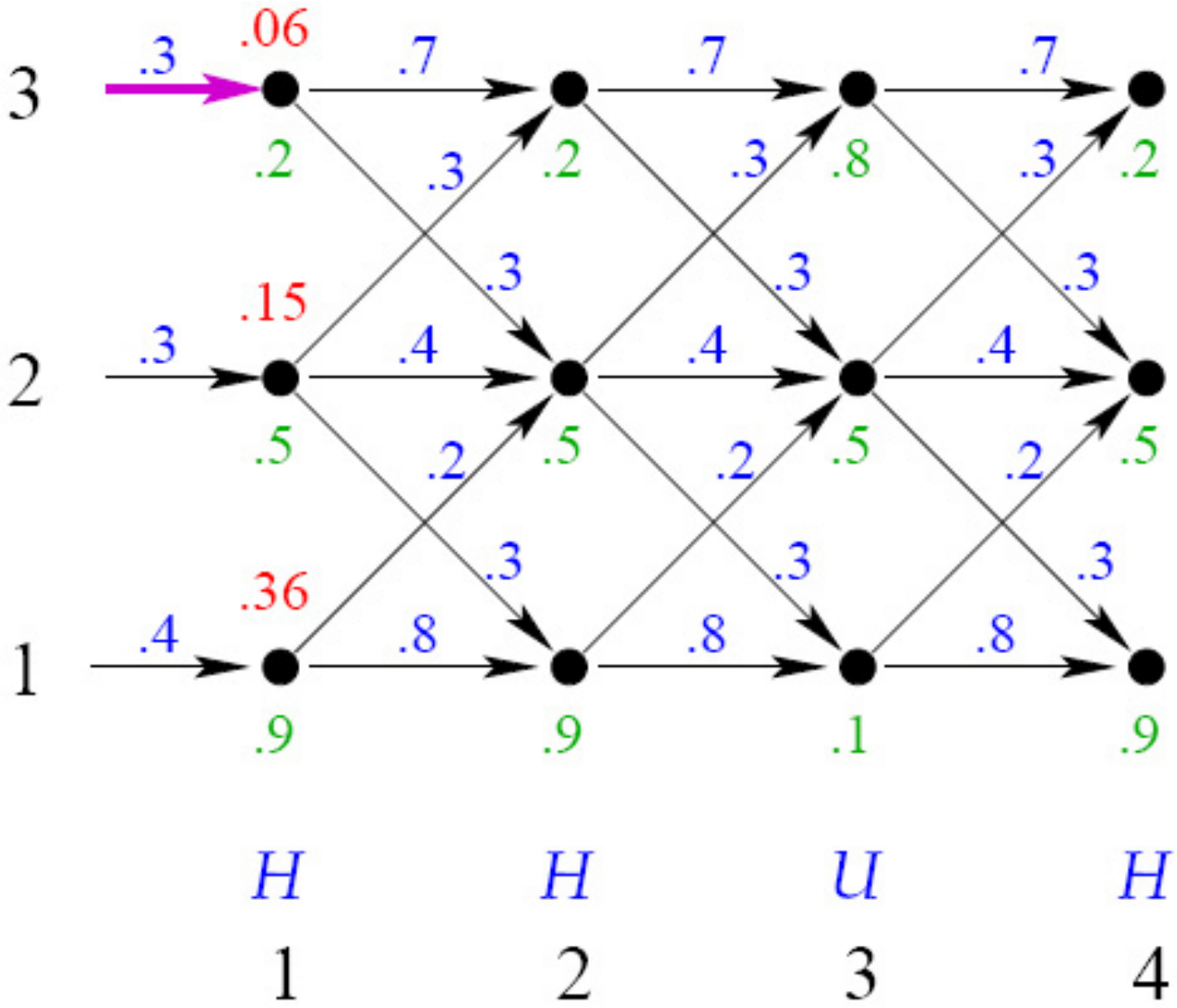
The forward algorithm: Example



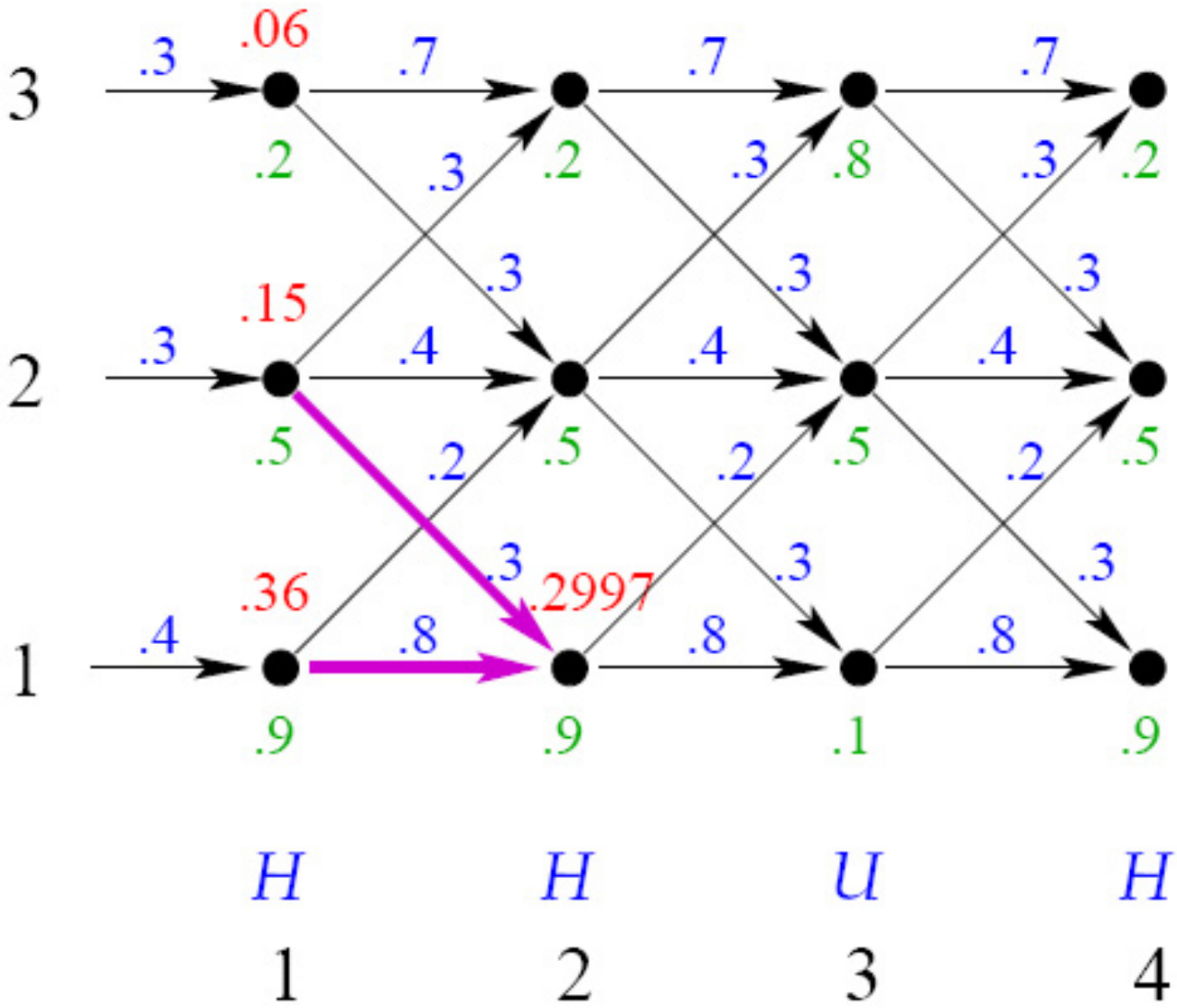
The forward algorithm: Example



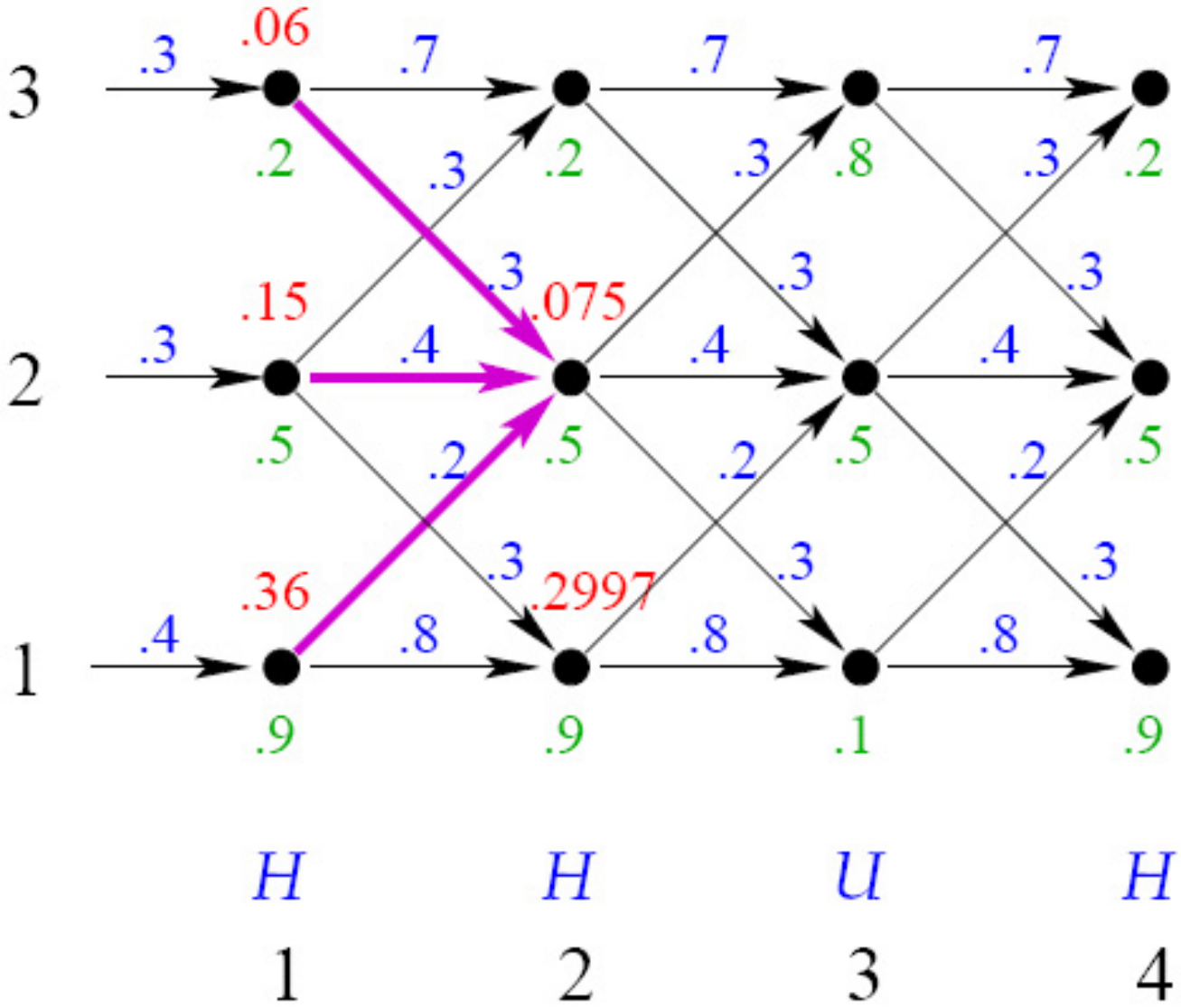
The forward algorithm: Example



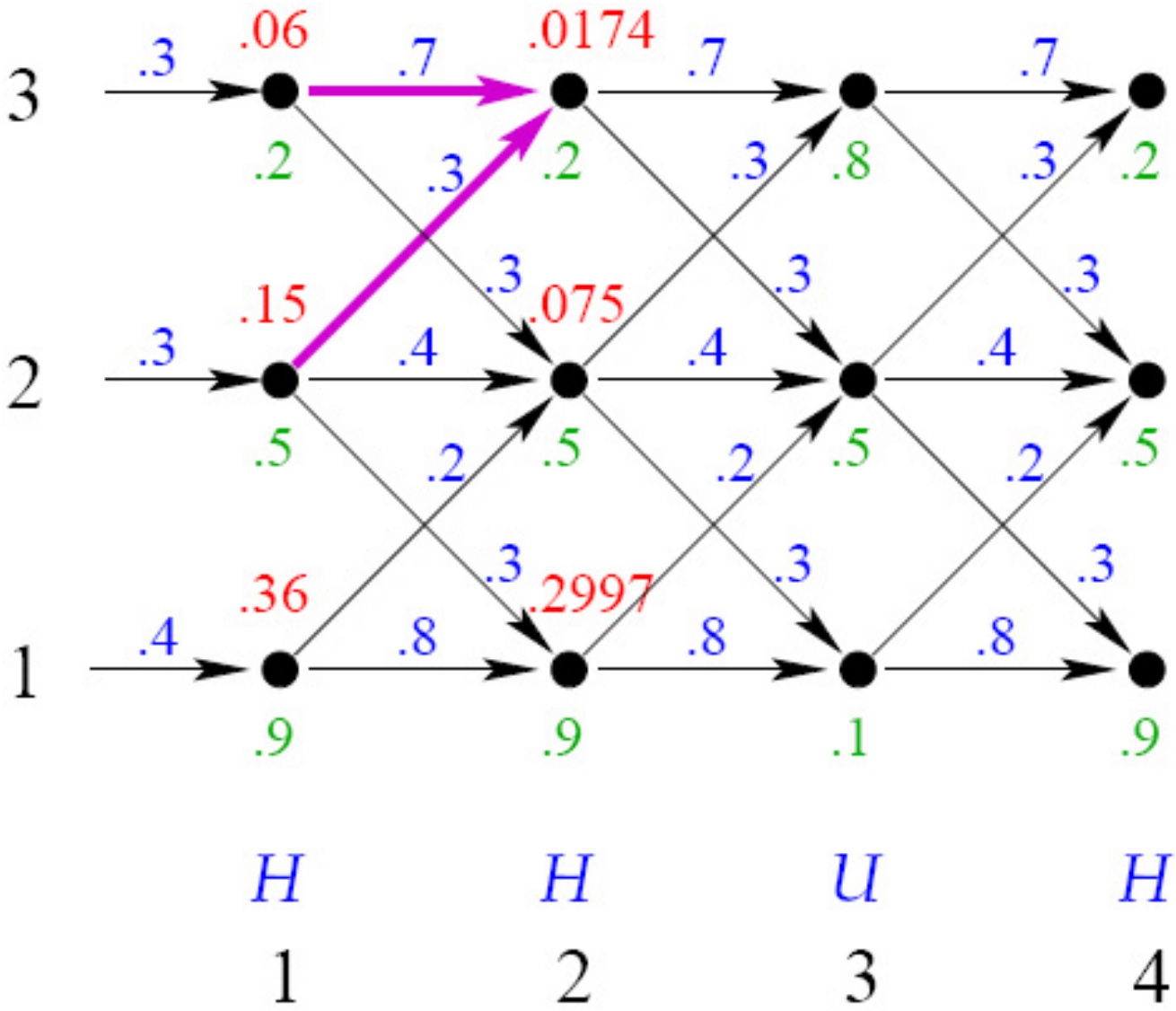
The forward algorithm: Example



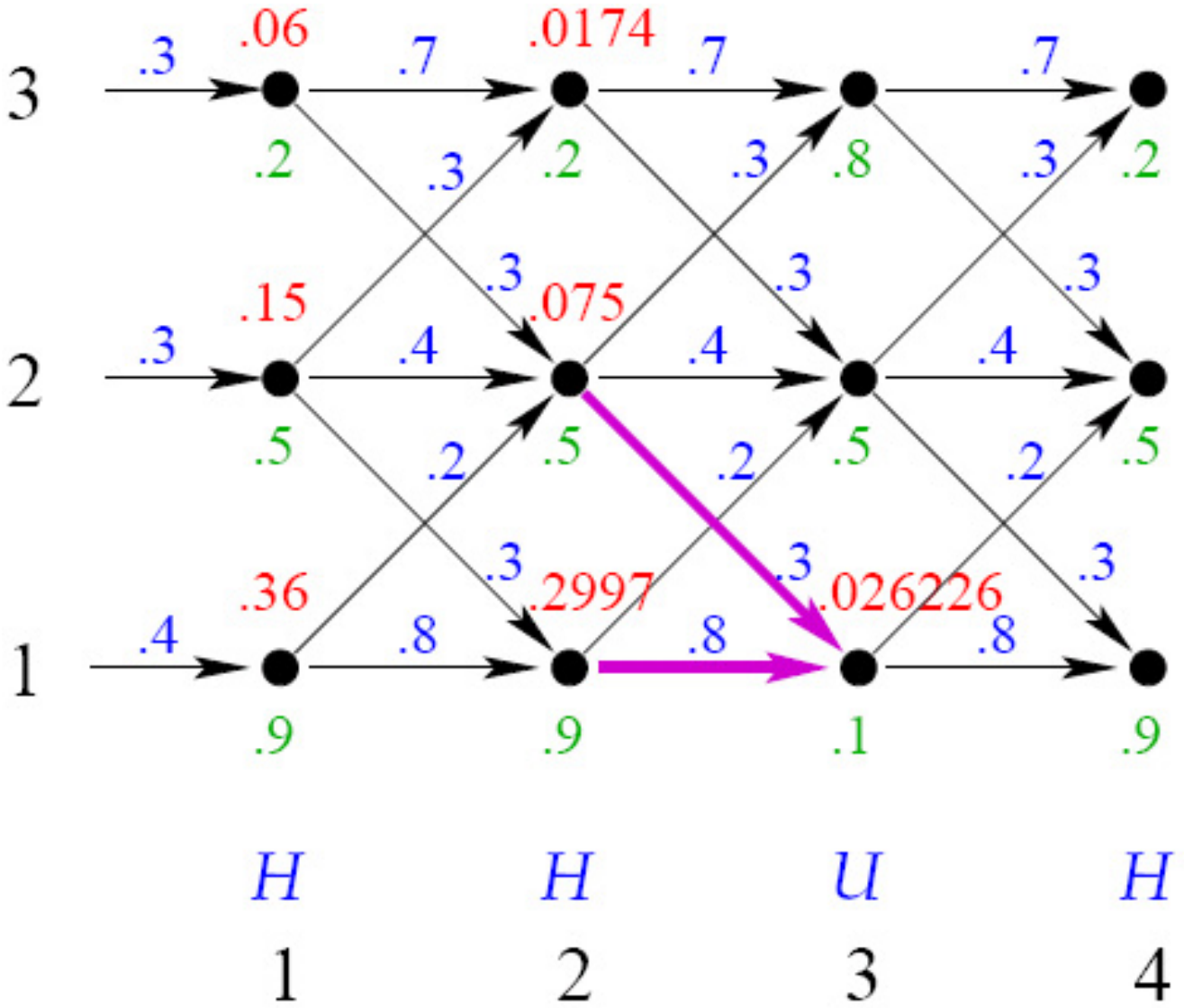
The forward algorithm: Example



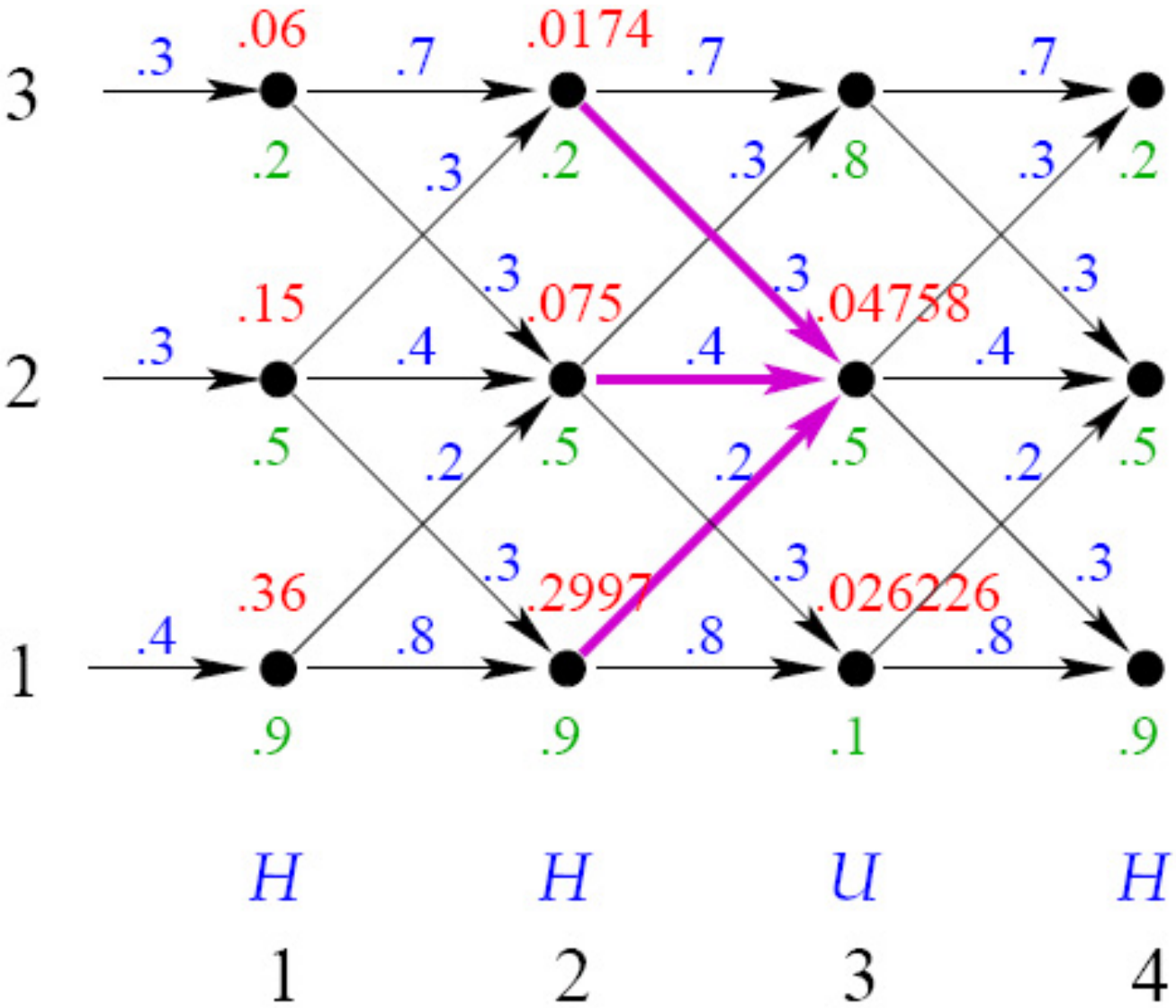
The forward algorithm: Example



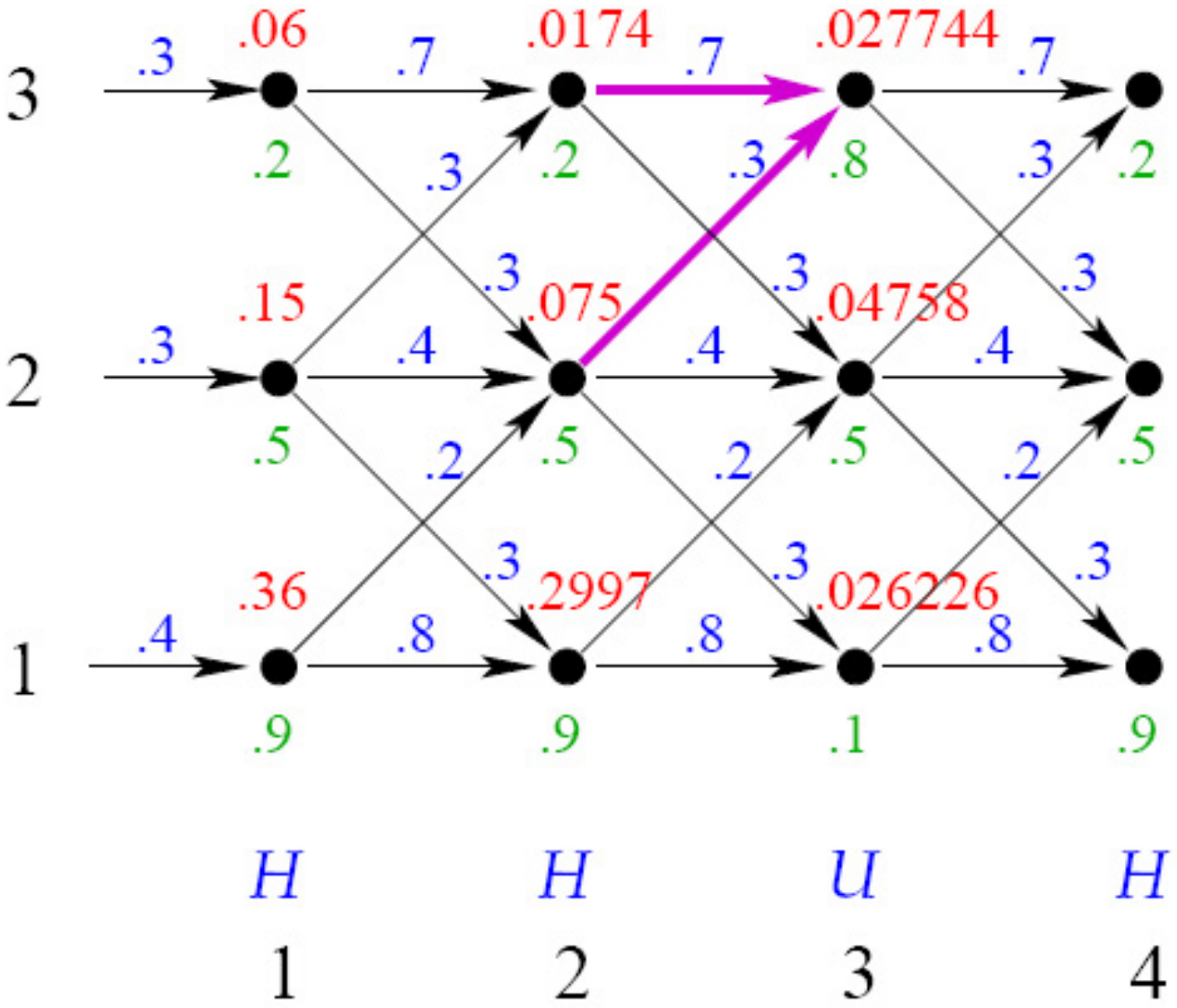
The forward algorithm: Example



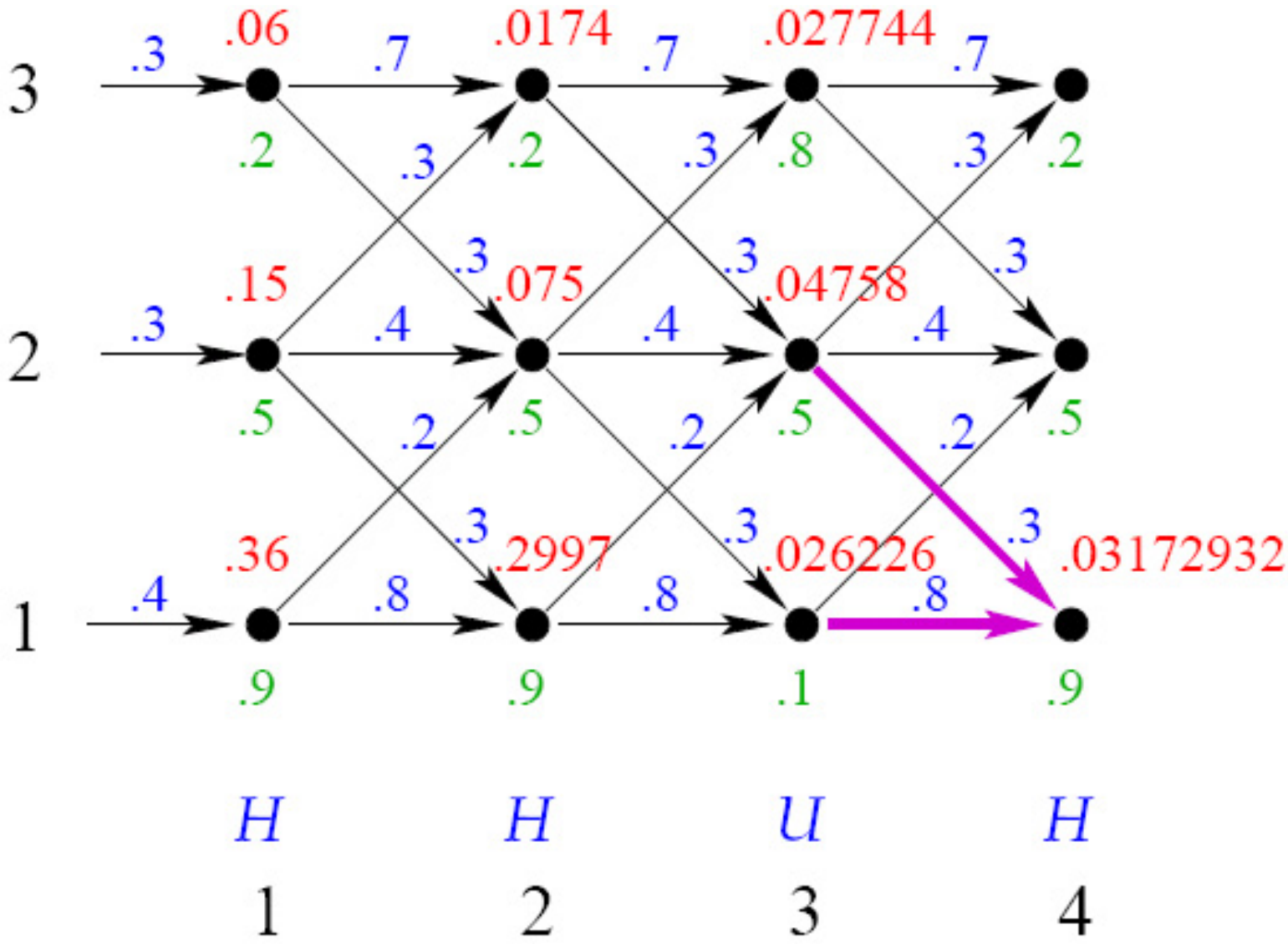
The forward algorithm: Example



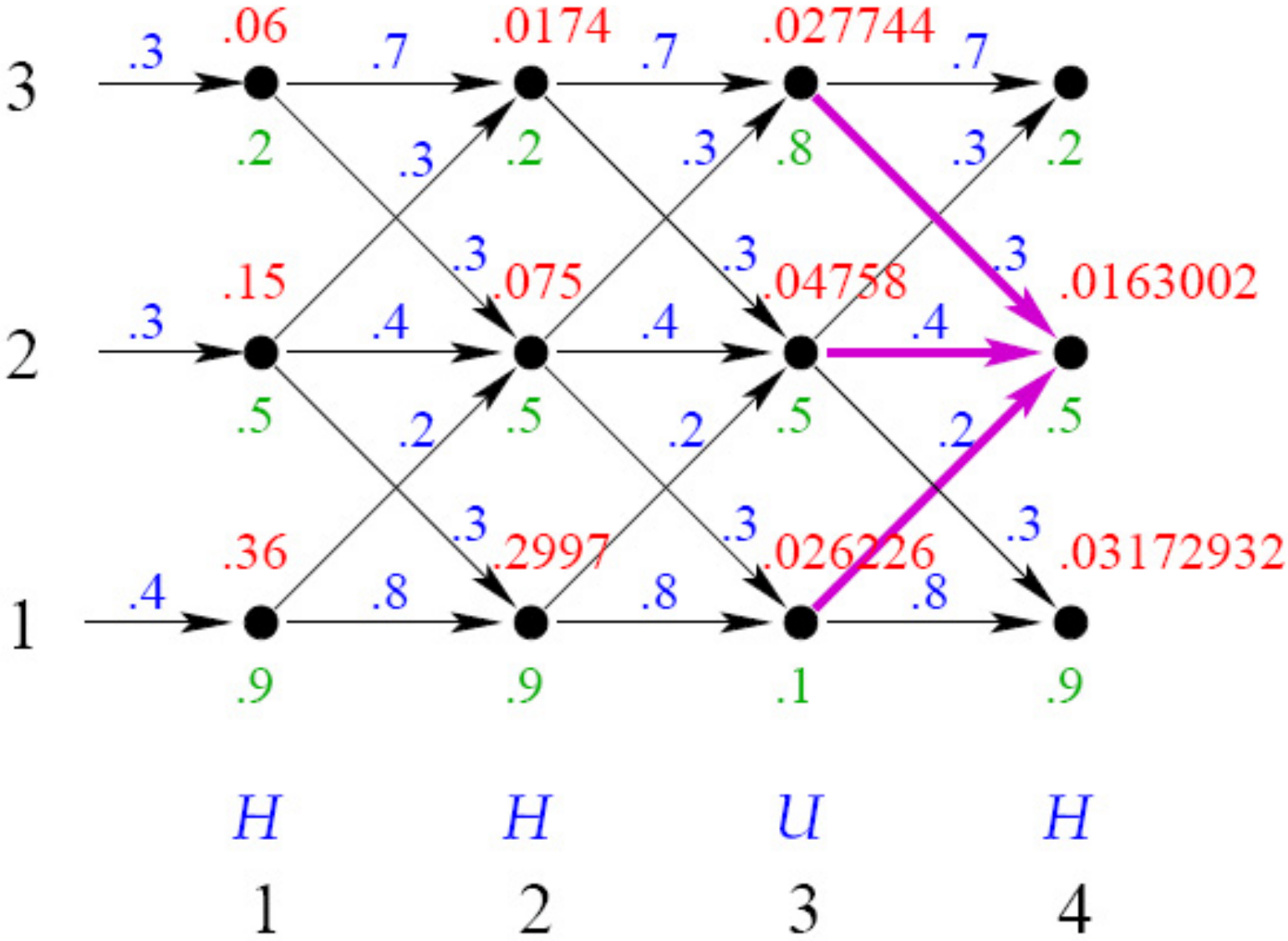
The forward algorithm: Example



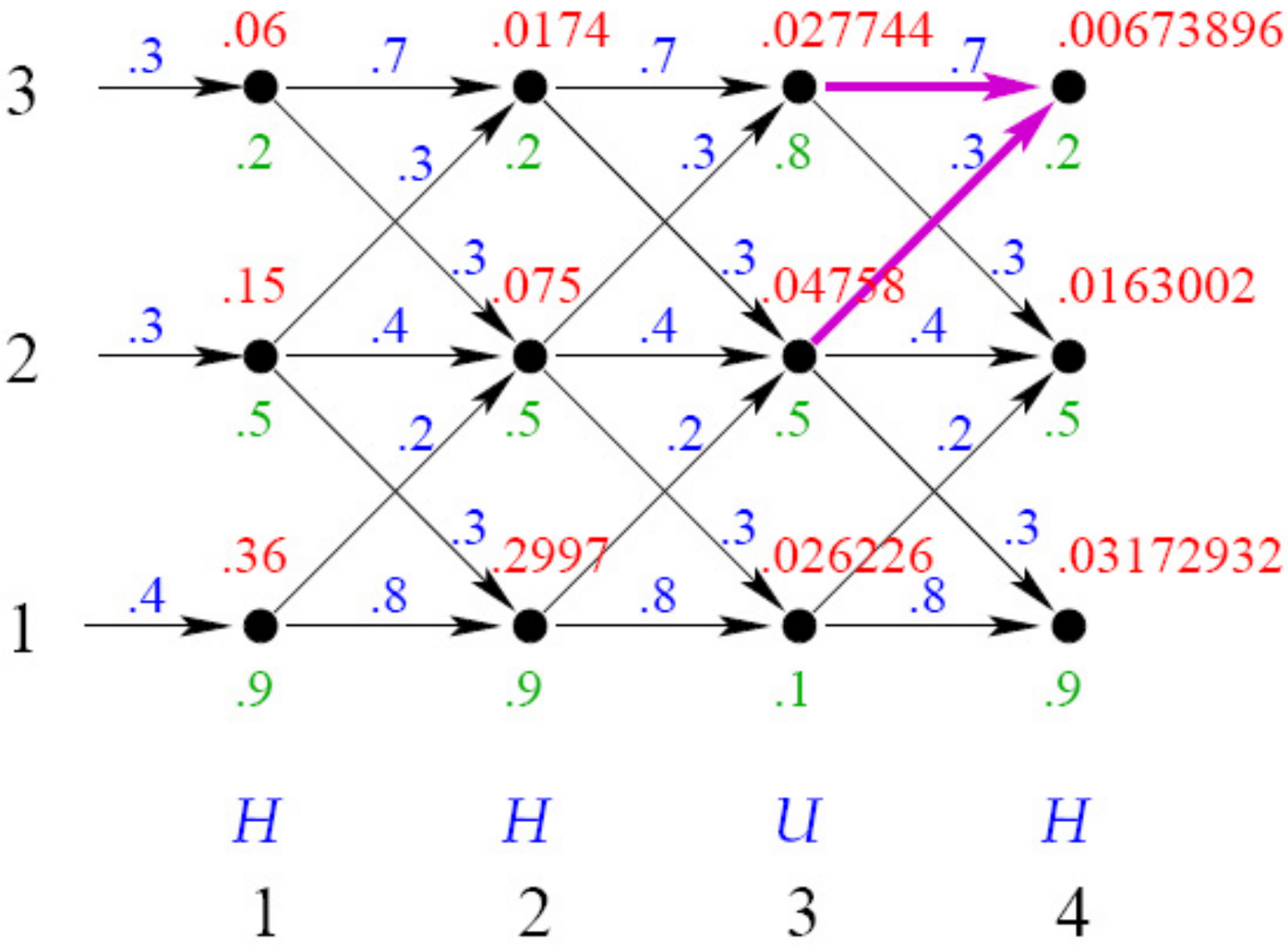
The forward algorithm: Example



The forward algorithm: Example



The forward algorithm: Example



Computing $p(O|\lambda)$: The backward algorithm

- Define the "backward" variable $\beta_t(i)$:

$$\beta_t(i) = P(\mathbf{o}_{t+1}\mathbf{o}_{t+2}\dots\mathbf{o}_T|q_t = i, \lambda), \quad \begin{array}{rcl} 1 \leq & t & \leq T-1, \\ 1 \leq & i & \leq N \end{array}$$

Computing $p(O|\lambda)$: The backward algorithm

- Define the "backward" variable $\beta_t(i)$:

$$\beta_t(i) = P(\mathbf{o}_{t+1}\mathbf{o}_{t+2}\ldots\mathbf{o}_T|q_t = i, \lambda), \quad \begin{array}{rcl} 1 \leq & t & \leq T-1, \\ 1 \leq & i & \leq N \end{array}$$

- Initialization: $\beta_T(i) = 1$

Computing $p(O|\lambda)$: The backward algorithm

- Define the "backward" variable $\beta_t(i)$:

$$\beta_t(i) = P(\mathbf{o}_{t+1}\mathbf{o}_{t+2}\ldots\mathbf{o}_T|q_t = i, \lambda), \quad \begin{array}{rcl} 1 \leq & t & \leq T-1, \\ 1 \leq & i & \leq N \end{array}$$

- Initialization: $\beta_T(i) = 1$
- Recursion: $\beta_t(i) = \sum_{j=1}^{N} a_{ij} b_j(\mathbf{o}_{t+1}) \beta_{t+1}(j)$

Computing $p(O|\lambda)$: The backward algorithm

- Define the "backward" variable $\beta_t(i)$:

$$\beta_t(i) = P(\mathbf{o}_{t+1}\mathbf{o}_{t+2}\ldots\mathbf{o}_T|q_t = i, \lambda), \quad \begin{array}{rcl} 1 \leq & t & \leq T-1, \\ 1 \leq & i & \leq N \end{array}$$

- Initialization: $\beta_T(i) = 1$
- Recursion: $\beta_t(i) = \sum_{j=1}^{N} a_{ij} b_j(\mathbf{o}_{t+1}) \beta_{t+1}(j)$
- Finally: $p(\mathbf{O}|\lambda) = \sum_{i=1}^{N} \pi_i b_i(\mathbf{o}_1) \beta_1(i)$

Computing $p(O|\lambda)$: The backward algorithm

- Define the "backward" variable $\beta_t(i)$:

$$\beta_t(i) = P(\mathbf{o}_{t+1}\mathbf{o}_{t+2}\dots\mathbf{o}_T|q_t = i, \lambda), \quad \begin{array}{rcl} 1 \leq & t & \leq T-1, \\ 1 \leq & i & \leq N \end{array}$$

- Initialization: $\beta_T(i) = 1$
- Recursion: $\beta_t(i) = \sum_{j=1}^{N} a_{ij} b_j(\mathbf{o}_{t+1}) \beta_{t+1}(j)$
- Finally: $p(\mathbf{O}|\lambda) = \sum_{i=1}^{N} \pi_i b_i(\mathbf{o}_1) \beta_1(i)$
- As in forward algorithm, computation is N^2T

Computing $p(O|\lambda)$: The backward algorithm

- Define the "backward" variable $\beta_t(i)$:

$$\beta_t(i) = P(\mathbf{o}_{t+1}\mathbf{o}_{t+2}\dots\mathbf{o}_T|q_t = i, \lambda), \quad \begin{array}{rcl} 1 \leq & t & \leq T-1, \\ 1 \leq & i & \leq N \end{array}$$

- Initialization: $\beta_T(i) = 1$
- Recursion: $\beta_t(i) = \sum_{j=1}^{N} a_{ij} b_j(\mathbf{o}_{t+1}) \beta_{t+1}(j)$
- Finally: $p(\mathbf{O}|\lambda) = \sum_{i=1}^{N} \pi_i b_i(\mathbf{o}_1) \beta_1(i)$
- As in forward algorithm, computation is N^2T
- Either algorithm alone can be used to compute $p(\mathbf{O}|\lambda)$, but both αs and βs will be needed for the training problem

CONGRATULATIONS!

- We've solved Problem 1!
- Questions? Comments?

Applications of HMMs: Speech processing

Automatic speech recognition

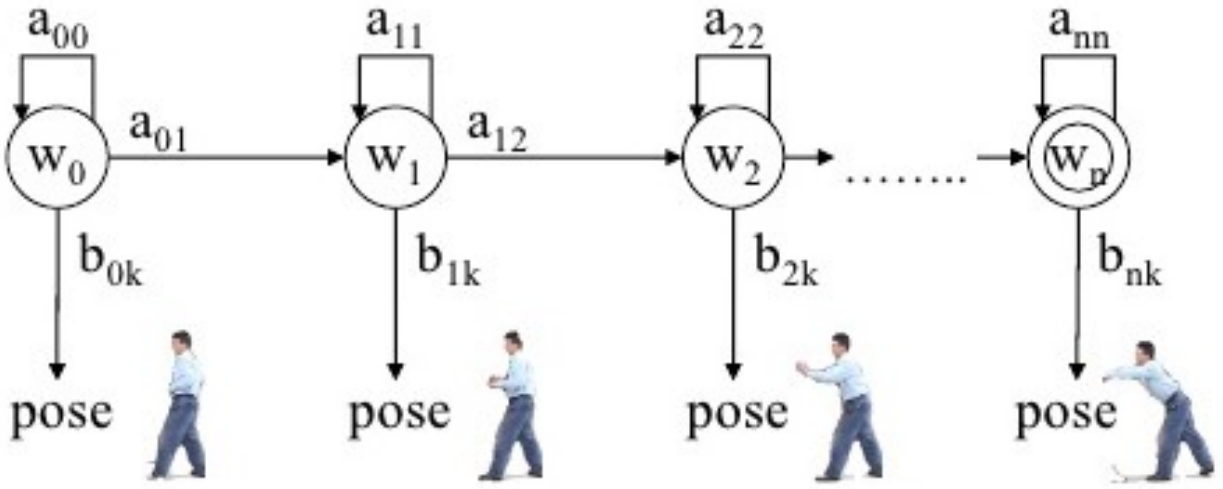
- One HMM per word or phoneme
- Time index corresponds to a 10ms "frame"
- Observation is a vector of spectral (frequency) measurements
- Can think of HMM state as corresponding to a state of the speaker's vocal tract

Unsupervised speech unit (word/phoneme) discovery

- Learn an HMM to model a collection of unlabelled speech
- Group together frequently occurring sequences of states to define units

Applications of HMMs: Activity recognition in video or biometrics

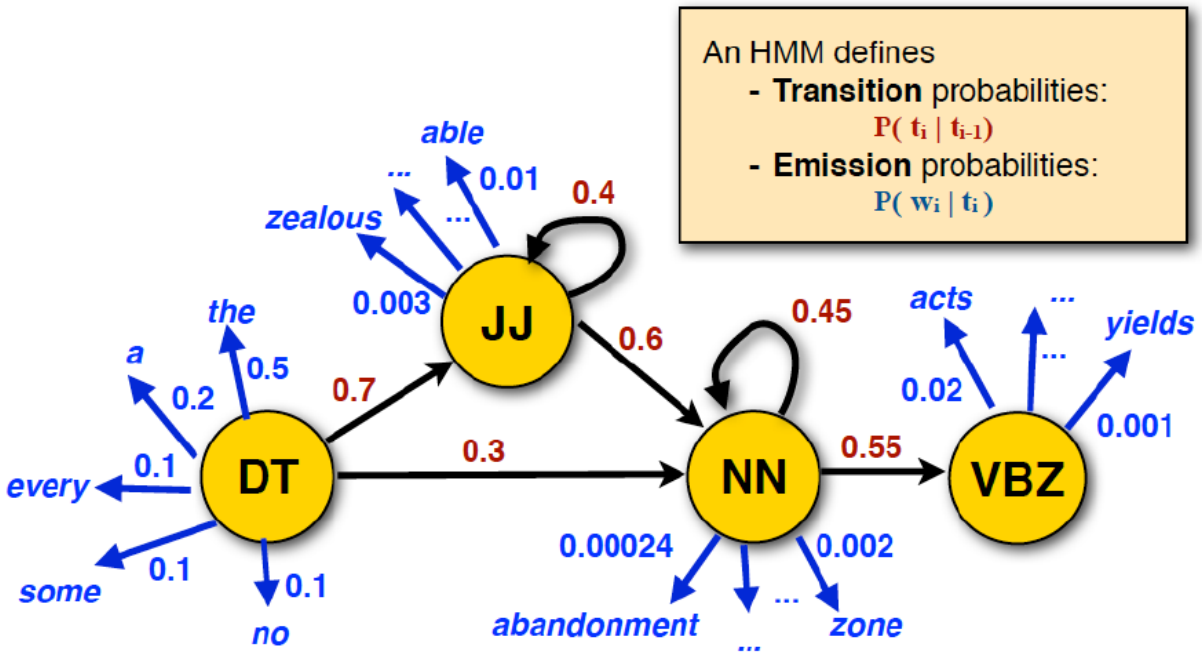
- State corresponds to "pose"
- As in speech recognition, the "activity" can be labeled or unlabeled



Applications of HMMs: Part-of-speech tagging

y2	y2	y1	y2	y1
Big	weak	stocks	are	bad
ADJ	ADJ	N	V	ADJ

- State corresponds to part of speech
- Sometimes used in fully supervised setting, sometimes unsupervised ("part of speech induction")



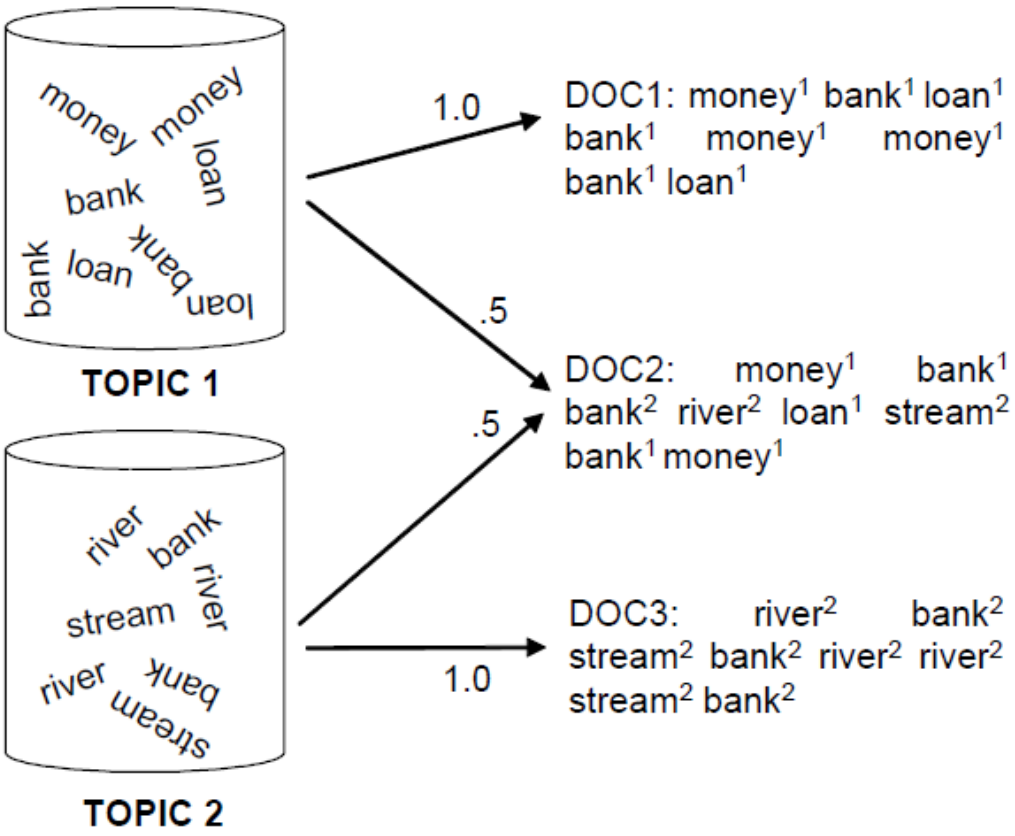
[Figure credit: Julia

Hidden topic Markov models (Gruber et al.)

- A type of HMM for topic modeling
- Extends the topic model of Steyvers & Griffiths and latent dirichlet allocation of Blei et al.

Reminder: Topic mixture models

- Documents are on *mixtures* of topics
- Each topic t has its own distribution over words $\boldsymbol{\beta}_t$
- A document d has its own distribution over topics $\boldsymbol{\theta}_d$ (multinomial over T topics)

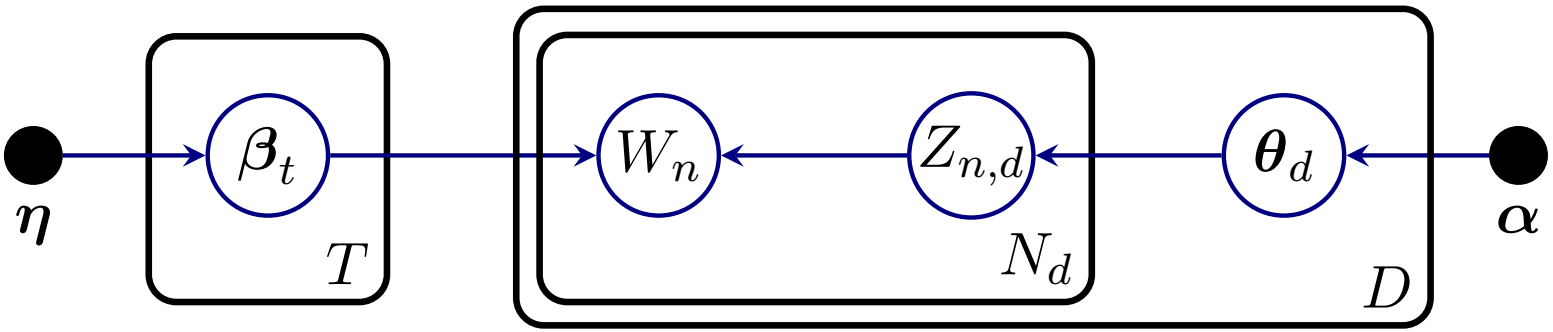


$$\boldsymbol{\theta}_1 = [1, 0]$$
$$\boldsymbol{\theta}_2 = [.5, .5]$$
$$\boldsymbol{\theta}_3 = [0, 1]$$

[Steyvers & Griffiths]

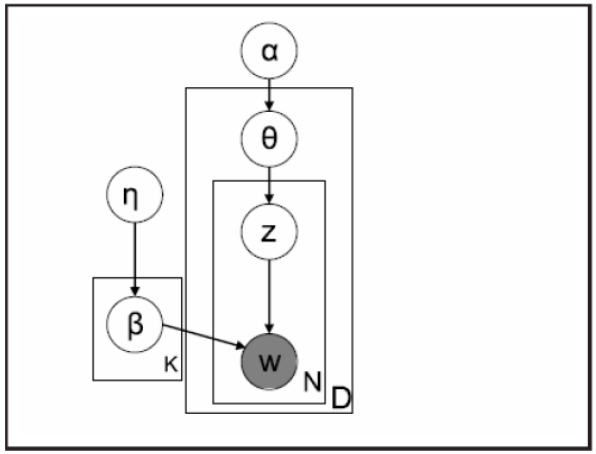
Topic model: generative process

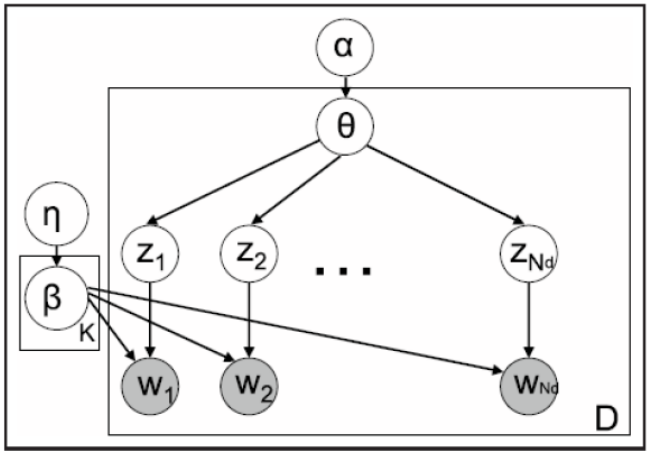
- D documents, document d with N_d words
- Model probability over topics for a document $\boldsymbol{\theta}_d$ with Dirichlet



- Recipe for generating a document with N words:
  (1) generate the set of T topics: draw each from Dirichlet($\boldsymbol{\eta}$)
  (2) generate $\boldsymbol{\theta}$: draw from Dirichlet($\boldsymbol{\alpha}$)
  (3) For each $n = 1, \ldots, N$ draw $Z_n \in \{0, \ldots, T\}$ from multinomial($\boldsymbol{\theta}$);
  then draw word W_n from multinomial($\boldsymbol{\beta}_{Z_n}$)

Topic mixture model: another view

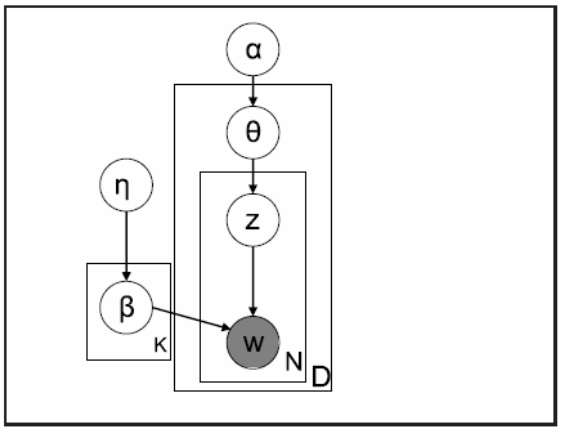


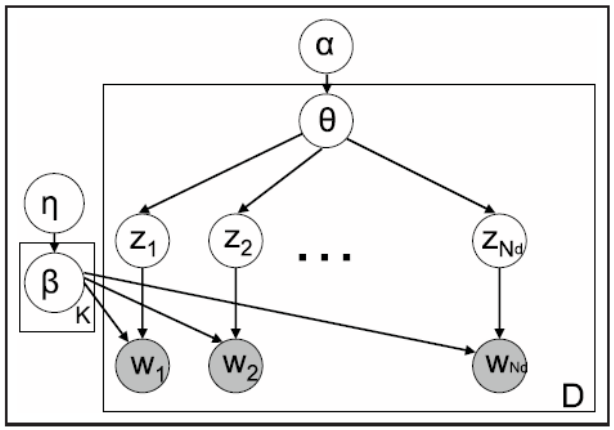


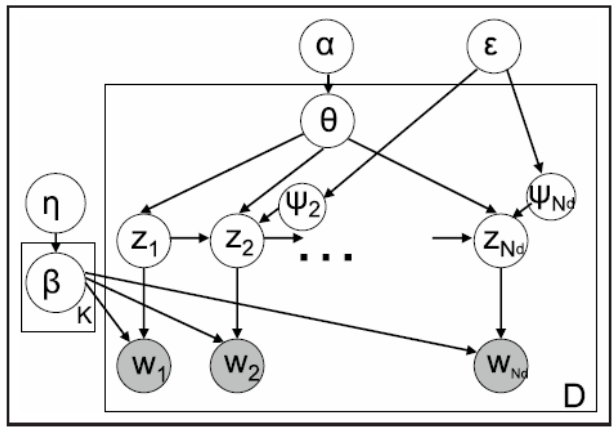
Hidden topic Markov model

Assumes that each sentence comes from a single topic

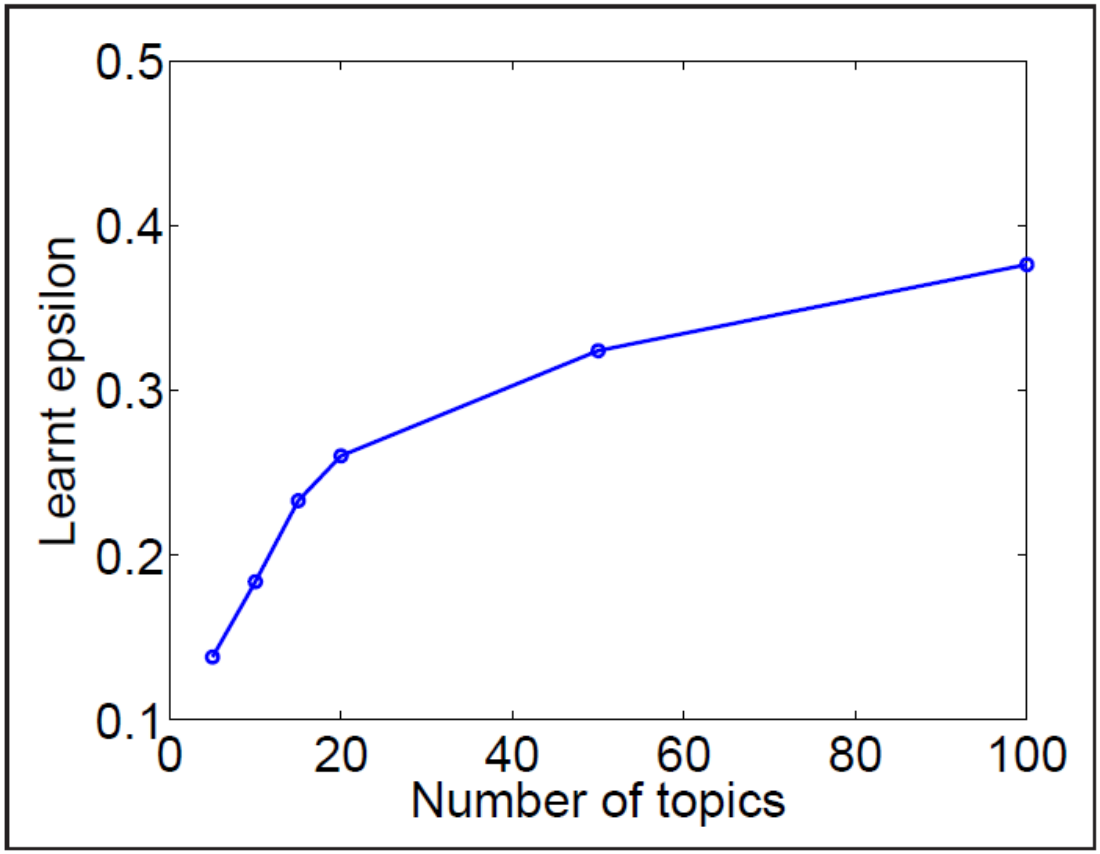
Probability ϵ of ending sentence and drawing a new topic



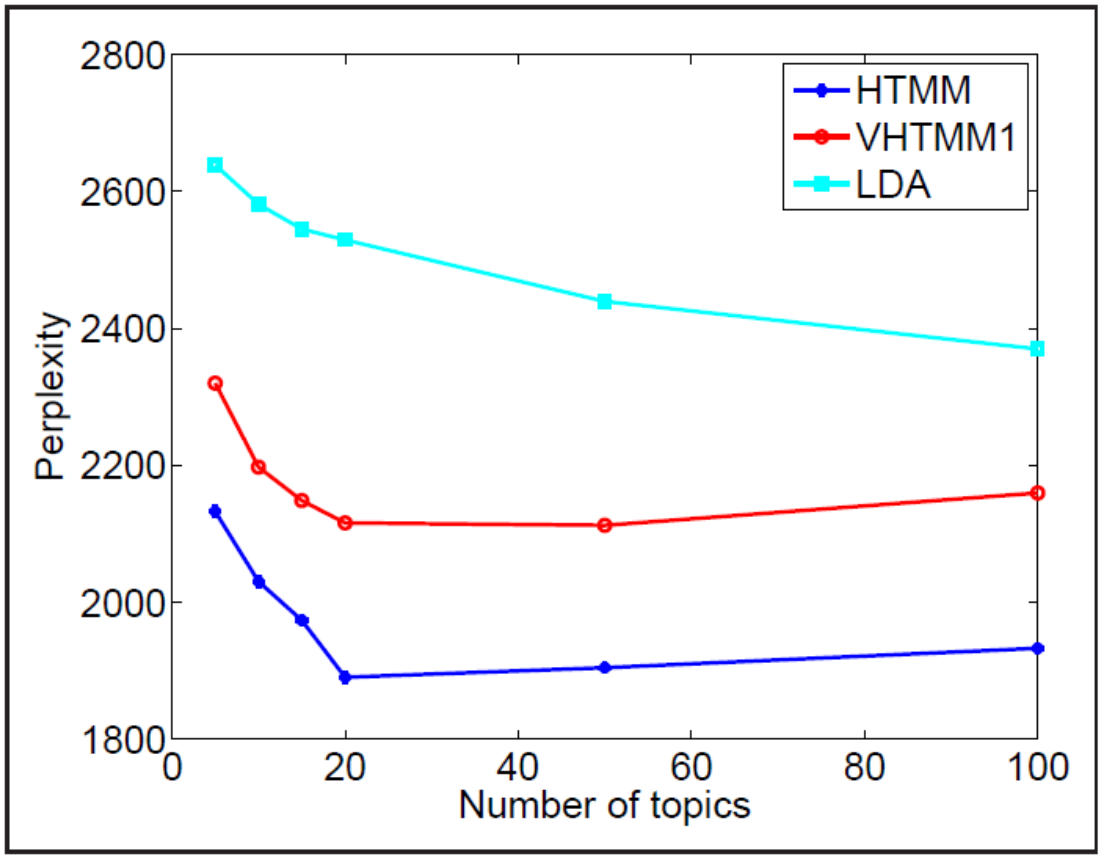




HTMM: The role of ϵ



HTMM vs. topic models: Perplexity results



HTMM results

Abstract We give necessary and sufficient conditions for uniqueness of the **support** vector solution for the problems of pattern recognition and regression estimation, for a general class of cost functions. We show that if the solution is not unique, all **support** vectors are necessarily at bound, and we give some simple examples of non-unique solu- tions. We note that uniqueness of the primal (dual) solution does not necessarily imply uniqueness of the dual (primal) solution. We show how to compute the threshold b when the solution is unique, but when all **support** vectors are at bound, in which case the usual method for determining b does not work.

recognition and regression estimation algorithms [12], with arbitrary convex costs, the value of the normal w will always be unique. Acknowledgments C. Burges wishes to thank W. Keasler, V. Lawrence and C. Nohl of Lucent Tech- nologies for their **support**. References [1] R. Fletcher. Practical Methods of Optimization. John Wiley and Sons, Inc., 2 nd edition, 1987.

Topic mixture model (smoothed LDA) results

Abstract We give necessary and sufficient conditions for uniqueness of the **support** vector solution for the problems of pattern recognition and regression estimation, for a general class of cost functions. We show that if the solution is not unique, all **support** vectors are necessarily at bound, and we give some simple examples of non-unique solu- tions. We note that uniqueness of the primal (dual) solution does not necessarily imply uniqueness of the dual (primal) solution. We show how to compute the threshold b when the solution is unique, but when all **support** vectors are at bound, in which case the usual method for determining b does not work.

recognition and regression estimation algorithms [12], with arbitrary convex costs, the value of the normal w will always be unique. Acknowledgments C. Burges wishes to thank W. Keasler, V. Lawrence and C. Nohl of Lucent Tech- nologies for their **support**. References [1] R. Fletcher. Practical Methods of Optimization. John Wiley and Sons, Inc., 2 nd edition, 1987.

Back to HMMs...

HMM problem 2: Decoding

- Given an observation sequence $\mathbf{O}$, what is the most likely state sequence $\mathbf{q}^* = \{q_1^*, \ldots, q_T^*\}$?
- Consider this criterion: Choose the states $\{q_1, \ldots, q_T\}$ that are *individually* most probable
- Define $\gamma_t(i)$ as the probability of being in state i at time t, given the observation sequence:
  $\gamma_t(i) = P(q_t = i|\mathbf{O}, \lambda)$
- Then the individually most likely state q_t at time t is:
  $q_t = \underset{1 \leq i \leq N}{\operatorname{argmax}} \gamma_t(i), \quad 1 \leq t \leq T$
- It is easy to show that $\gamma_t(i) = \frac{\alpha_t(i)\beta_t(i)}{p(\mathbf{O}|\lambda)}$

HMM problem 2: Decoding (2)

- The "individually most likely" criterion maximizes the expected number of correct states
- But ...

HMM problem 2: Decoding (2)

- The "individually most likely" criterion maximizes the expected number of correct states
- But ... it can give very unlikely (or impossible) state sequences

HMM problem 2: Decoding (2)

- The "individually most likely" criterion maximizes the expected number of correct states
- But ... it can give very unlikely (or impossible) state sequences
- Another criterion: Choose the state sequence that maximizes $p(\mathbf{q}|\mathbf{O}, \lambda)$ (or, equivalently, that maximizes $p(\mathbf{q}, \mathbf{O}|\lambda)$
- This can be done via the Viterbi algorithm
  - A dynamic programming algorithm

The Viterbi algorithm

- Define $\delta_t(i)$ as the highest probability along a single path to state i at time t that accounts for the first t observations:
  $\delta_t(i) = \max_{q_1,\cdots,q_{t-1}} P(q_1 q_2 \ldots q_t = i, \mathbf{o}_1 \mathbf{o}_2 \ldots \mathbf{o}_t | \lambda)$

The Viterbi algorithm

- Define $\delta_t(i)$ as the highest probability along a single path to state i at time t that accounts for the first t observations:
  $\delta_t(i) = \max_{q_1,\ldots,q_{t-1}} P(q_1 q_2 \ldots q_t = i, \mathbf{o}_1 \mathbf{o}_2 \ldots \mathbf{o}_t | \lambda)$
- Initialization: $\delta_1(i) = \pi_i b_i(\mathbf{o}_1)$ (Note: this $= \alpha_1(i)$)

The Viterbi algorithm

- Define $\delta_t(i)$ as the highest probability along a single path to state i at time t that accounts for the first t observations:
  $\delta_t(i) = \max_{q_1,\dots,q_{t-1}} P(q_1 q_2 \dots q_t = i, \mathbf{o}_1 \mathbf{o}_2 \dots \mathbf{o}_t | \lambda)$
- Initialization: $\delta_1(i) = \pi_i b_i(\mathbf{o}_1)$ (Note: this $= \alpha_1(i)$)
- Recursion: $\delta_t(j) = \max_{1 \le i \le N} [\delta_{t-1}(i) a_{ij}] \, b_j(\mathbf{o}_t), \quad 2 \le t \le T$
- Note the similarity to the α recursion:
  $$\alpha_t(j) = \left[\sum_{i=1}^{N} \alpha_{t-1}(i) a_{ij} \right] b_j(\mathbf{o}_t)$$
  - Same as the forward algorithm, except we keep the max rather than the sum at each node!

The Viterbi algorithm

- Define $\delta_t(i)$ as the highest probability along a single path to state i at time t that accounts for the first t observations:
  $\delta_t(i) = \max_{q_1,\cdots,q_{t-1}} P(q_1 q_2 \ldots q_t = i, \mathbf{o}_1 \mathbf{o}_2 \ldots \mathbf{o}_t | \lambda)$
- Initialization: $\delta_1(i) = \pi_i b_i(\mathbf{o}_1)$ (Note: this $= \alpha_1(i)$)
- Recursion: $\delta_t(j) = \max_{1 \le i \le N} [\delta_{t-1}(i) a_{ij}] \, b_j(\mathbf{o}_t), \quad 2 \le t \le T$
- Note the similarity to the α recursion:
  $$\alpha_t(j) = \left[\sum_{i=1}^{N} \alpha_{t-1}(i) a_{ij} \right] b_j(\mathbf{o}_t)$$
  - Same as the forward algorithm, except we keep the max rather than the sum at each node!
  - To retrieve the best path, also need to keep a back-pointer from each node to the best incoming partial path
- As in forward algorithm, computation is $\approx N^2 T$

CONGRATULATIONS!

- We've solved Problem 2!
- Questions? Comments?

Solving the training problem for HMMs

- The problem: Given an observation sequence (training data), find the HMM parameters $\lambda^* = \{\pi^*, \mathbf{A}^*, \mathbf{B}^*\}$ that maximize the probability of the observations

$$\lambda^* = \underset{\lambda}{\operatorname{argmax}}\, p(\mathbf{O}|\lambda)$$

- If someone *gave us* the state sequence, it would be easy to find the maximum-likelihood estimates for $\mathbf{A}$ and $\mathbf{B}$:

states	1	1	1	1	2	3	3	2	3	2	2	1
observations	H	H	U	H	H	U	U	U	H	H	H	H

$$a_{ij} = \frac{\text{count}(i \rightarrow j)}{\text{count}(\text{transitions from } i)}$$

$$b_i(k) = \frac{\text{count}(v_k \text{ in state } i)}{\text{count}(i)}$$

Solving the training problem for HMMs (2)

- We are *not* given state alignments. Instead we will "generate" them ourselves (as we did with Gaussian component labels in GMMs)

- Baum-Welch Algorithm: EM for HMMs
  - Make initial guess of parameter values, then alternate between
  - Compute the posterior probabilities of all states at all times, given the observations
  - Compute the maximum-likelihood parameters given the *expected* numbers of occurrences of states and transitions

The Baum-Welch algorithm

- Define $\xi_t(i,j)$ as the probability of being in state i at time t and state j at time $t+1$, given the observation sequence

$$\begin{aligned}\xi_t(i,j) &= P(q_t = i, q_{t+1} = j | \mathbf{O}, \lambda) \\ &= \frac{\alpha_t(i) a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)}{p(\mathbf{O}|\lambda)}\end{aligned}$$

The Baum-Welch algorithm

- Define $\xi_t(i,j)$ as the probability of being in state i at time t and state j at time $t+1$, given the observation sequence

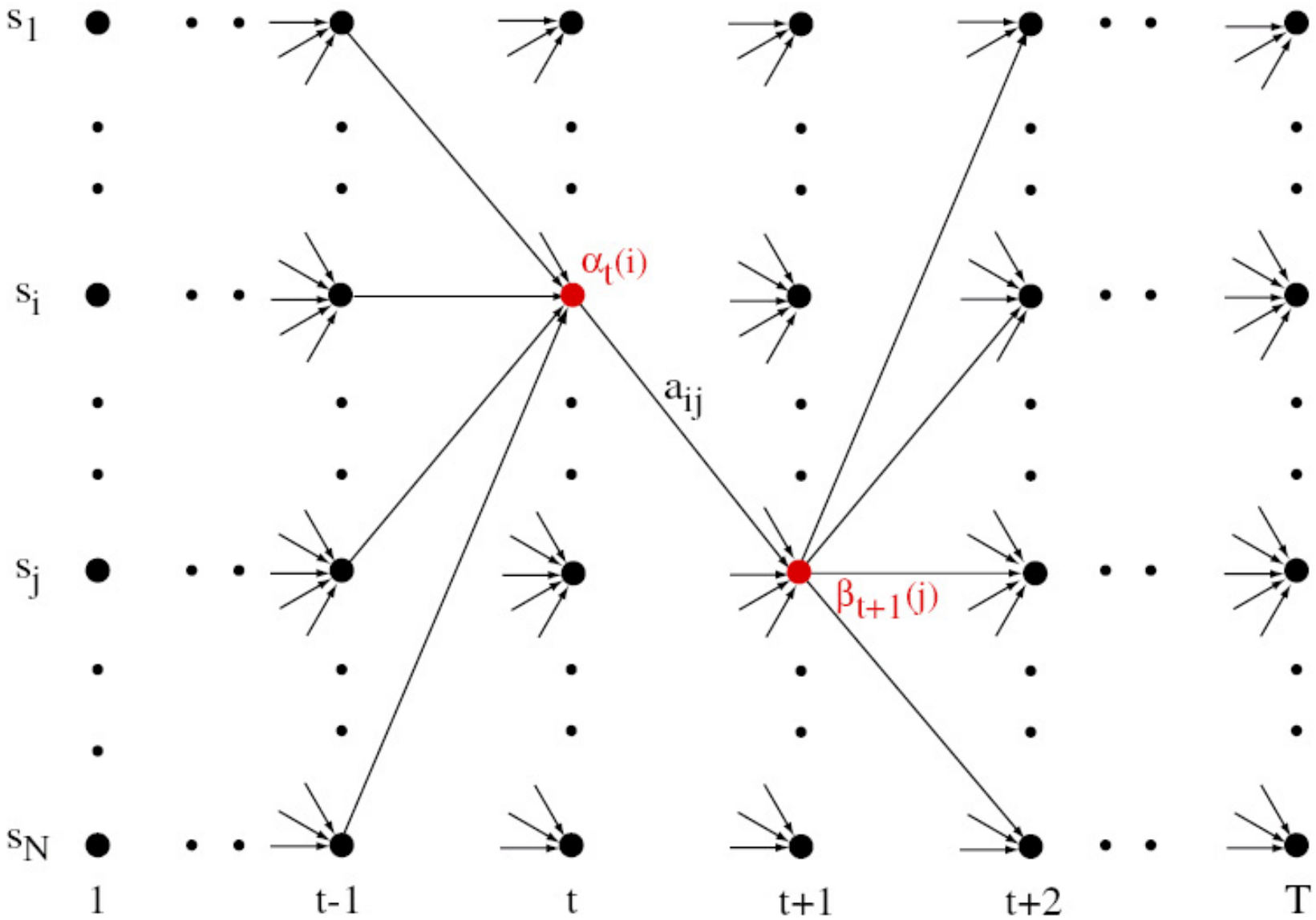
$$\begin{aligned}\xi_t(i,j) &= P(q_t = i, q_{t+1} = j|\mathbf{O}, \lambda) \\ &= \frac{\alpha_t(i)a_{ij}b_j(o_{t+1})\beta_{t+1}(j)}{p(\mathbf{O}|\lambda)}\end{aligned}$$

- Then note that the state posterior probabilities are $\gamma_t(i) = P(q_t = i|\mathbf{O}, \lambda) = \sum_{j=1}^{N} \xi_t(i,j)$
- Now we can compute the *expected counts* we need, e.g.:

$$\sum_{t=1}^{T-1} \gamma_t(i) = \text{expected count of transitions from state } i$$

$$\sum_{t=1}^{T-1} \xi_t(i,j) = \text{expected count of transitions from } i \text{ to } j$$

The Baum-Welch algorithm



The Baum-Welch re-estimation formulas

$$\hat{a}_{ij} = \frac{\text{expected-count}(i \rightarrow j)}{\text{expected-count}(\text{transitions from } i)} = \frac{\sum_{t=1}^{T-1} \xi_t(i,j)}{\sum_{t=1}^{T-1} \gamma_t(i)}$$

$$\hat{b}_i(k) = \frac{\text{expected-count}(v_k \text{ in state } i)}{\text{expected-count}(i)} = \frac{\sum_{t=1, o_t=v_k}^{T} \gamma_t(i)}{\sum_{t=1}^{T} \gamma_t(i)}$$

$$\hat{\pi}_i = \text{expected-count}(q_1 = i) = \gamma_1(i)$$

Properties of Baum-Welch re-estimation

- If λ is the current model, and $\hat{\lambda}$ is the model after one iteration of re-estimation, then it can be shown that $p(\mathbf{O}|\hat{\lambda}) \geq p(\mathbf{O}|\lambda)$
- I.e., the likelihood is increased at each iteration
- Therefore, we can iterate the re-estimation formulas until some convergence threshold is reached (e.g. likelihood increase less than some threshold τ)
- Guaranteed to find a *local* maximum of the likelihood, but not a global one

Parameter estimation with EM

- Maximum-likelihood parameter estimation algorithms for GMMs and HMMs are examples of the more general *expectation-maximization* algorithm:
  - Initialize parameters with a guess, then iterate until convergence:
  - Given current parameter estimates, compute posteriors of latent variables
  - Using posteriors as "counts", find maximum-likelihood estimates of parameters

Independence properties in HMMs

An HMM is a model of the joint distribution of $\mathbf{q}$ and $\mathbf{O}$:

$$
\begin{aligned}
p(\mathbf{O}, \mathbf{q}) = p(\mathbf{O}_{1:T}, \mathbf{q}_{1:T}) &= p(q_1)p(\mathbf{o}_1|q_1)\prod_{t=2}^{T} p(q_t|q_{t-1})p(\mathbf{o}_t|q_t) \\
&= \pi_{q_1} b_{q_1}(\mathbf{o}_q)\prod_{t=2}^{T} a_{q_{t-1}q_t} b_{q_t}(\mathbf{o}_t)
\end{aligned}
$$

What independence properties does this imply?

Independence properties in HMMs

An HMM is a model of the joint distribution of $\mathbf{q}$ and $\mathbf{O}$:

$$
\begin{aligned}
p(\mathbf{O}, \mathbf{q}) = p(\mathbf{O}_{1:T}, \mathbf{q}_{1:T}) &= p(q_1)p(\mathbf{o}_1|q_1)\prod_{t=2}^{T} p(q_t|q_{t-1})p(\mathbf{o}_t|q_t) \\
&= \pi_{q_1} b_{q_1}(\mathbf{o}_q) \prod_{t=2}^{T} a_{q_{t-1}q_t} b_{q_t}(\mathbf{o}_t)
\end{aligned}
$$

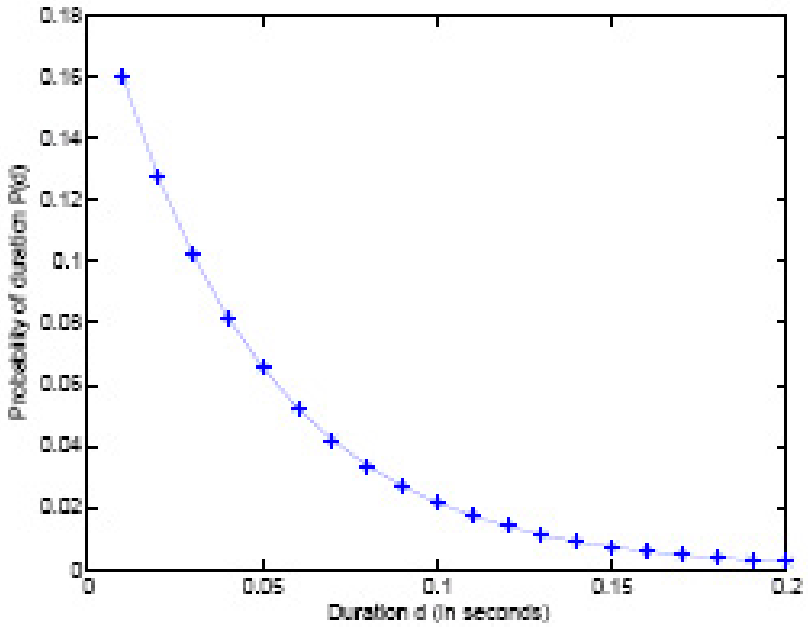
What independence properties does this imply?

- Notation: $X \perp \{Y, Z\}|W \Rightarrow X$ is independent of Y and Z given W
- $\mathbf{o}_t \perp \{\mathbf{q}_{\neg t}, \mathbf{O}_{\neg t}\}|q_t$, where $\mathbf{q}_{\neg t} = \{q_1, \ldots, q_{t-1}, q_{t+1}, \ldots, q_T\}$, i.e. given the current state, the current observation depends on nothing else
- $\{\mathbf{q}_{1:t-1}, \mathbf{O}_{1:t-1}\} \perp \{\mathbf{q}_{t+1:T}, \mathbf{O}_{t+1:T}\}|q_t$, i.e. the future is independent of the past given the current state

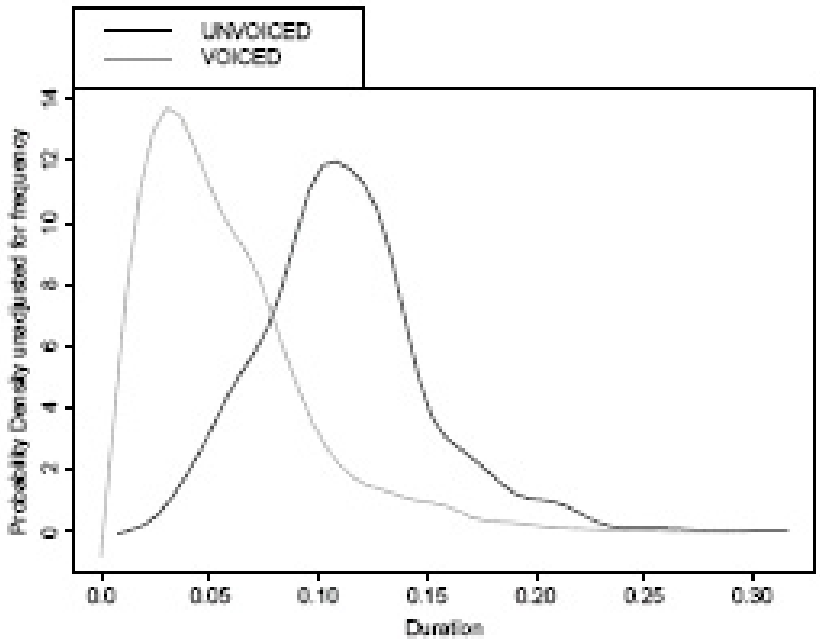
HMM extensions: State duration modeling

- The probability of staying in state i for d time steps (frames) is $a_{ii}^{d-1}(1 - a_{ii})$, a geometric distribution
- In contrast, durations of "states" we'd like to model often look nothing like that – example of phoneme durations:

HMM state duration probabilities, $a_{ii} = .8$



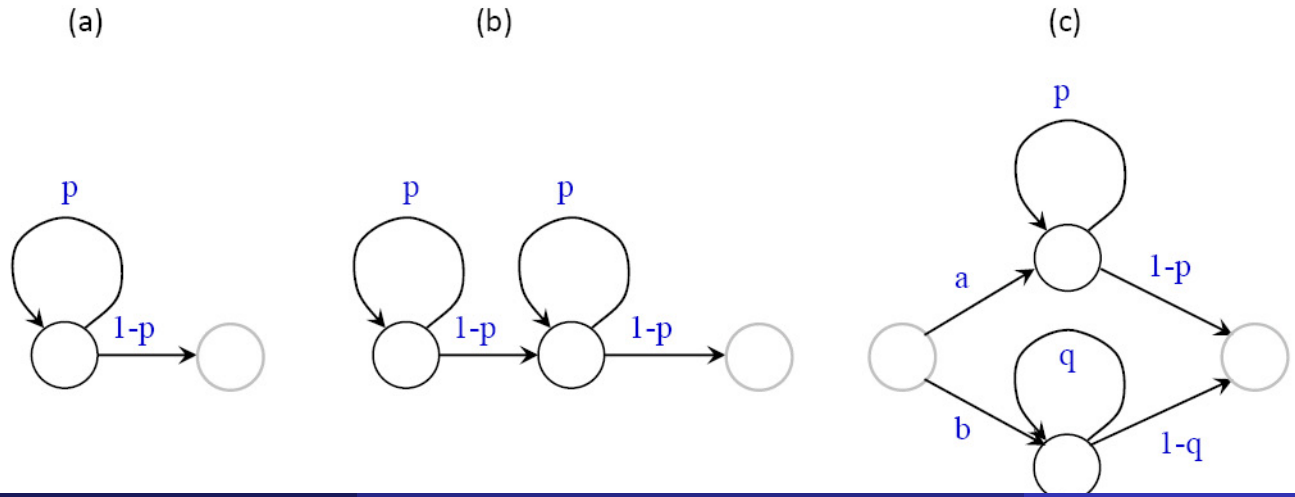
Actual fricative duration probabilities



State duration modeling

Possible solutions:

- Replace transition probabilities with explicit duration distributions [Ostendorf '96]
  - Most general approach
  - Breaks dynamic programming algorithms
- Replace single state with multiple *tied states* with identical observation distribution, and some desired transition structure
  - Simplest option: Set a minimum duration
  - Can simulate other duration distributions with alternative transition structures



HMMs with continuous observation distributions

- In a *continuous-density* HMM (CD-HMM), the discrete observation probabilities, $b_i(k)$, are replaced by continuous densities $b_i(\mathbf{o})$
- E.g., the observation distribution can be a Gaussian or mixture of Gaussians: $b_i(\mathbf{o}) = \sum_{k=1}^{K} c_{ik} \mathcal{N}(\mathbf{o}|\mu_{ik}, \Sigma_{ik}), \quad 1 \leq i \leq N$
- $c_{ik}, \mu_{ik}, \Sigma_{ik}$ are the parameters of Gaussian component k in state i
- Alternatively, observation distribution can be modelled by a neural network (referred to as "hybrid HMM/NN" in speech recognition literature)
- Doesn't change anything in the algorithms; just replace $b_i(k)$ by the value of the corresponding density